

UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZE TEORICHE E APPLICATE

CORSO DI STUDIO TRIENNALE IN
INFORMATICA

Sviluppo di uno strumento di migrazione dati Infor LN

Relatore:
Prof. Massazza Paolo

Tesi di Laurea di:
Gabriele Caretti
Matricola 756564

Anno accademico:
2025/2026

Indice

1 Introduzione **1**

1.1 Contesto Aziendale 1

1.2 Obiettivi del Progetto 1

1.3 Metodologia e Strumenti 2

1.4 Contesto Tecnologico e Curva di Apprendimento 3

1.5 Struttura della Tesi 4

2 Stato Attuale **5**

2.1 Sistemi ERP 5

2.1.1 Evoluzione dei Sistemi Legacy al Cloud 5

2.1.2 Architetture Multitenant e Segregazione dei Dati 5

2.1.3 Confronto con Altre Piattaforme ETL 6

2.2 Infor LN e CloudSuite Enterprise 7

2.2.1 Linguaggio Baan C: 3GL e 4GL 9

2.2.2 Infor Exchange e Strumenti di Import/Export 10

2.3 Migrazione Dati nei Sistemi ERP 10

2.3.1 Il Paradigma ETL (Extract, Transform, Load) 10

2.3.2 Sfide e Problematiche Comuni 12

2.3.3 Necessità di Strumenti di Automazione e Generazione Dinamica 12

2.4 Limiti delle Tecnologie Alternative 13

3 Analisi dei Requisiti **14**

3.1 Scenario di Riferimento 14

3.1.1 Necessità di Ottimizzazione 14

3.2 Requisiti Funzionali 14

3.2.1 Sviluppo dell'Importer per Infor Exchange 15

3.2.2 Estensibilità con Funzioni Ad-hoc 16

3.2.3 Integrazione con lo Strumento Documentale 16

3.3 Requisiti Non Funzionali 16

3.3.1 Manutenibilità ed Estensibilità 16

3.3.2 Sicurezza e Data Integrity in Multitenancy 16
3.3.3 Usabilità (UX/UI) 17
3.4 Casi d'Uso (Use Cases) 17
3.5 Modello dei Dati 20
3.5.1 Analisi Strutture Dati Coinvolte 20

4 Architettura **23**
4.1 Panoramica Architettura di Sistema 23
4.1.1 Modello Dati e Dizionario 23
4.2 Progettazione del Software 25
4.2.1 Design dell'Importer e del Parser dei Dati 25
4.2.2 Sequenza Operativa del Sistema 25
4.2.3 Diagramma dei Componenti 26
4.2.4 Generazione del Codice 26
4.2.5 Code Generation vs. Interpretation 27
4.2.6 Modulo di Integrazione Ad-hoc e Callbacks 28
4.3 Scelte Tecnologiche 28
4.3.1 Linguaggi e Framework 28
4.3.2 Librerie di Supporto e Strumenti di Build 28

5 Implementazione **29**
5.1 Sviluppo dell'Importer Infor Exchange 29
5.1.1 Gestione dei Formati e Parsing 29
5.1.2 Ricostruzione delle Definizioni dei File 30
5.2 Implementazione del Generatore di Codice 32
5.2.1 Creazione Dinamica del Codice 32
5.2.2 Funzioni di Trasformazione Ad-hoc 34
5.2.3 Gestione delle Latenze e Fallback Dinamico 35
5.2.4 Logging degli Errori 37
5.2.5 Gestione delle Latenze Transazionali 38
5.3 Interfacce Utente (Sessioni) 39
5.3.1 Design dell'Interfaccia Utente e Scelte UX/UI 39
5.3.2 Esecuzione e Monitoraggio 41
5.3.3 Librerie (DLL) e Data Access Layer (DAL) 42
5.3.4 Tabelle di Sistema e Sessioni Operative 42
5.4 Integrazione Documentale 44

5.4.1 Esportazione delle Specifiche di Migrazione 44
5.5 Ciclo di Vita del Dato 45

6 Testing e Validazione **46**
6.1 Strategia di Testing 46
6.1.1 Unit Testing sui Moduli di Parsing 46
6.1.2 Test del Flusso di Migrazione 46
6.1.3 Test della Sicurezza 47
6.2 Verifica dell'Integrità dei Dati 49
6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud) 49
6.3 Analisi delle Prestazioni e ROI 49
6.3.1 Benchmark delle Prestazionali 49
6.3.2 Analisi degli Errori 50
6.3.3 Return on Investment (ROI) 51

7 Conclusioni **53**
7.1 Obiettivi e Risultati Raggiunti 53
7.2 Sfide Incontrate 53
7.3 Sviluppi Futuri 54

A Dettaglio della Struttura delle Tabelle **56**
A.1 Tabelle di Configurazione e Metadati 56
A.2 Tabelle di Mapping e Logica 58
A.3 Tabelle di Esecuzione e Log 60

Bibliografia **62**

Sitografia **63**

Capitolo 1

Introduzione

1.1 Contesto Aziendale

Questo lavoro di tesi nasce dalla necessità dell'azienda AlteaIN di sviluppare e migliorare uno strumento di migrazione dati per Infor LN [13], un sistema *ERP* (*Enterprise Resource Planning*) industriale utilizzato a livello internazionale, di cui l'azienda è ufficialmente partner in Italia.

L'obiettivo è agevolare il processo di migrazione da sistemi *legacy* a Infor LN, garantendo l'integrità e la coerenza dei dati in ambienti *multitenant* [12, 10].

Nello specifico, questo strumento sarà sviluppato all'interno dell'azienda, parte del gruppo di Altea Federation, che offre servizi Infor in Italia (Figura 1.1). AlteaIN è specializzata nella consulenza informatica, e si occupa di supportare le aziende del comparto manifatturiero e logistico nei loro processi di trasformazione digitale e innovazione tecnologica.



(a) Logo AlteaIN (b) Logo Infor

Figura 1.1: Loghi di AlteaIN e Infor, rispettivamente azienda ospitante e fornitore del software.

1.2 Obiettivi del Progetto

Il lavoro si concentrerà sull'analisi dei requisiti, la progettazione dell'architettura del software, la stesura della documentazione e lo sviluppo di un prototipo funzionante.

In particolare, lo strumento sarà in grado di generare dinamicamente codice, estendibile con funzioni ad hoc, per migrare dati su sistemi *Cloud Multitenant*, con un focus sulla Infor *CloudSuite Enterprise*. Questa generazione permetterà di abbattere i tempi di sviluppo di *script* personalizzati per ciascuna migrazione (noti anche come operazioni di *Porting*), riducendo i classici errori delle scritture manuali e aumentandone l'efficienza operativa.

Inoltre, è richiesto lo sviluppo di un *importer* capace di interpretare il formato proprietario di *import/export* di Infor *Exchange* al fine di ricreare automaticamente le definizioni fisiche dei file dall'origine da migrare. L'intero sistema dovrà interfacciarsi e integrarsi con lo strumento aziendale preesistente per la produzione di specifiche di migrazione e la relativa esportazione documentale.

1.3 Metodologia e Strumenti

Essendo il *Migration Tool* destinato a diventare uno strumento molto utile e utilizzato dai consulenti AlteaIN, lo sviluppo ha richiesto alti requisiti di qualità, seguendo le *best practice* dell'ingegneria del software [8], sotto la supervisione del tutor aziendale (*Manager*).

Durante il periodo di tirocinio, le attività sono state gestite ispirandosi ai princìpi *Agile* [9], adottando un approccio iterativo e incrementale. Il ciclo di sviluppo è stato suddiviso in *Sprint* di durata bisettimanale (variabile in base alla fase del progetto), al termine dei quali veniva rilasciata e testata una nuova funzionalità o un modulo specifico del *Migration Tool*.

Il lavoro è stato scandito da momenti di confronto regolari, come *stand-up meeting* giornalieri e *Sprint Review* periodiche con il tutor aziendale. Questi incontri hanno permesso di allineare costantemente lo sviluppo ai requisiti e alle regole imposte dall'architettura Infor. Il *Product Backlog*, contenente l'elenco delle funzionalità da implementare, dei bug da risolvere e delle ottimizzazioni architetturali, è stato gestito attivamente per rispondere tempestivamente alle necessità del progetto.

Per il tracciamento delle attività, l'analisi dei requisiti e la stesura della documentazione tecnica, si è fatto affidamento su strumenti di collaborazione e *project management* aziendali come software di *ticketing*, garantendo così un al-

lineamento costante e trasparente tra la figura dello sviluppatore (il sottoscritto) e i consulenti funzionali.

I consulenti, in quanto futuri utenti finali del *tool*, hanno giocato un ruolo chiave nel definire gli *Use Case* e nel testare le *release* intermedie del generatore di codice.

1.4 Contesto Tecnologico e Curva di Apprendimento

Uno degli aspetti più sfidanti e formativi del tirocinio, preliminare alla fase di sviluppo vera e propria, è stato l'inserimento in un ecosistema software proprietario ed estremamente verticalizzato. A differenza dei linguaggi *open-source* o dei *framework* convenzionali largamente studiati durante il percorso accademico (come Java o *framework web*), l'ambiente Infor si basa su un'architettura consolidata denominata *Infor Tools*.

Il linguaggio di programmazione impiegato è il **Baan C** (nelle sue versioni 3GL e 4GL), un linguaggio procedurale ed *event-driven* strettamente legato al dizionario dati (*Data Dictionary*) del database sottostante (che può essere di diversi tipi, a noi non noti per motivi riservati). Questa peculiarità ha richiesto un importante periodo iniziale di *training* e studio della documentazione ufficiale (come *progguide* e videolezioni registrate create per colleghi e clienti), necessario per padroneggiare concetti avanzati come le *Dynamic Link Libraries* (DLL) proprietarie, i *Data Access Layer* (DAL), la gestione della memoria in un contesto *Enterprise* e l'utilizzo degli strumenti dedicati. Solo a valle di questa significativa curva di apprendimento è stato possibile ideare e implementare i moduli architetturali che compongono il *Migration Tool*.

Tra questi elementi si possono citare:

- **Architettura**: Infor segue un'architettura a più livelli, da quelli fisici, database, a quelli di comunicazione con l'esterno tramite *middleware* ION API [11] (in fase di transizione da XML con SOAP a REST [2]), includendo *web service*, *BOD* (*Business Object Documents*) e nuovi strumenti.

- **Strumenti di sviluppo**: Strumenti di sviluppo come IDE proprietari (basati su IDE come, per esempio, *Eclipse*) LN Studio, BECS e interfaccia web LNUI.
- **Librerie e moduli**: L'ecosistema Infor è composto da numerose librerie e moduli preesistenti, che vanno dalle *utility* di stringa alle librerie di accesso ai dati, fino a moduli specifici per la gestione dei processi di *business*.

1.5 Struttura della Tesi

L'elaborato si articola nel seguente modo:

- **Capitolo 2**: stato attuale degli ERP moderni, architetture cloud *multitenant* e sfide di migrazione dati.
- **Capitolo 3**: analisi dello scenario di partenza e dei requisiti funzionali e non funzionali.
- **Capitolo 4**: architettura del sistema, design e scelte tecnologiche (*Importer* e Generatore codice).
- **Capitolo 5**: descrizione dell'implementazione concreta del progetto.
- **Capitolo 6**: testing, inclusi elementi come la validazione dell'integrità dei dati e statistiche.
- **Capitolo 7**: conclusioni, traguardi raggiunti e possibili sviluppi futuri.

Capitolo 2

Stato Attuale

2.1 Sistemi ERP

I sistemi *ERP* (*Enterprise Resource Planning*) rappresentano un elemento fondamentale del sistema gestionale aziendale [15], consentendo l'integrazione dei processi di *business* principali di un'azienda come: vendite, acquisti, gestione inventario, contabilità e personale.

2.1.1 Evoluzione dei Sistemi Legacy al Cloud

Nel corso degli anni, i sistemi *ERP* si sono evoluti partendo da architetture *on-premise legacy*, spesso caratterizzate da un elevato grado di personalizzazione (*customization*, in Infor LN definiti come *package* `tx`) e importanti costi di manutenzione. Tuttavia, oggi si sta assistendo ad una transizione verso nuove soluzioni come il *Cloud*.

2.1.2 Architetture Multitenant e Segregazione dei Dati

Di fatto, il modello *Cloud Multitenant* oggi rappresenta l'architettura fondamentale degli *ERP* moderni [12]. A differenza dei tradizionali sistemi *on-premise* (definiti *single-tenant*), in cui ogni azienda possiede un'infrastruttura hardware e software dedicata, in un ambiente *multitenant* una singola istanza dell'applicativo e del database viene condivisa da molteplici clienti (i cosiddetti *tenant*, in ambiente *cloud*).

Questo approccio offre molti vantaggi dal punto di vista dell'erogazione del servizio (scalabilità orizzontale, aggiornamenti continui e massivi, ottimizzazione delle risorse e abbassamento dei costi di manutenzione dei server), ma pone grandi sfide architetturali. La principale problematica risiede nell'isolamento

logico delle informazioni: essendo i dati fisicamente residenti nello stesso database, è fondamentale che siano divisi (segregazione) per evitare violazioni di sicurezza e fughe di dati (*data leak*) tra aziende.

Nei sistemi come Infor *CloudSuite Enterprise*, la segregazione dei dati viene applicata a più livelli.

- A livello di infrastruttura *Cloud*, l'isolamento del *tenant* è garantito da chiavi di crittografia specifiche e partizionamenti logici.
- A livello applicativo, la storicizzazione e l'isolamento avvengono tramite l'uso sistematico di identificativi aziendali (spesso indicati come *Company Number* o codici azienda) che vengono inseriti implicitamente dal *framework* in ogni *query* verso il database.

Da questa architettura deriva la necessità che ogni processo di migrazione operi secondo vincoli di integrità estremamente rigidi [14]. Qualsiasi inserimento massivo di dati deve necessariamente passare attraverso strati intermedi di validazione (come i *Data Access Layer*) che assicurino che il dato venga assegnato non solo alla tabella e con il formato corretto, ma anche e soprattutto all'identificativo del *tenant* di destinazione, senza possibilità di errore o sovrascrittura accidentale.

2.1.3 Confronto con Altre Piattaforme ETL

Sebbene sul mercato esistano soluzioni ETL di terze parti ampiamente consolidate (come Talend o Informatica PowerCenter), la loro integrazione con l'*ERP Infor CloudSuite* presenta grandi barriere architetturali. L'utilizzo di questi *tool*, pur offrendo interfacce grafiche avanzate per la gestione dei *workflow*, comporta infatti i seguenti svantaggi:

- **Latenza di rete:** dovuta alla continua interazione tra l'ERP e i servizi di integrazione esterni.
- **Accesso limitato ai dati:** impossibilità di interagire nativamente con il *Data Dictionary* del sistema.

- ***Bottleneck* nelle interfacce:** obbligo di effettuare le conversioni fuori dall'ecosistema ed elaborare le scritture a sistema appoggiandosi esclusivamente ad API predefinite o ai *BOD (Business Object Document)*.
- **Costi:** necessità di stipulare costosi accordi di licenza.

Per superare questi limiti strutturali e garantire un'elevata efficienza transazionale, si è scelto uno sviluppo interno di un proprio *tool*. Il *Migration Tool* realizzato sfrutta un approccio di *Metaprogramming* implementato nativamente in Baan C. Questa scelta architetturale permette di operare interamente all'interno dell'ecosistema Infor, evitando intermediari esterni e gestendo le regole di business e i controlli di referenza direttamente dall'interno.

2.2 Infor LN e CloudSuite Enterprise

Infor LN, nato con il nome di *Baan ERP*, è uno dei principali *ERP* nel settore manifatturiero industriale [13] a livello internazionale. Viene integrato all'interno della Infor *CloudSuite Enterprise*, il quale offre funzionalità per la gestione della *supply chain* e produzione [10].

L'architettura logica e fisica di Infor LN si basa su un classico ma solido modello a tre livelli (Figura 2.1), volto a disaccoppiare interamente l'interazione utente, la logica di *business* e l'accesso al database.

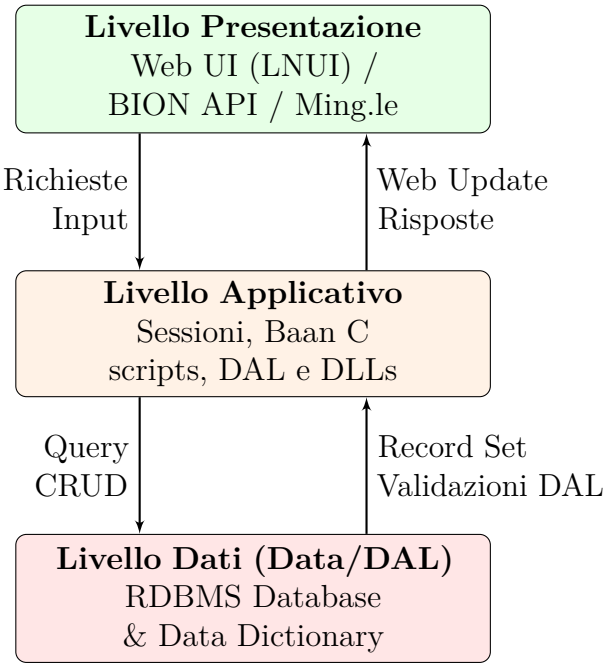


Figura 2.1: Rappresentazione schematica dell'architettura a 3 livelli (Three-tier) di Infor LN.

L'interfaccia utente Web (nota come `LNUI`) permette agli utenti finali di navigare tra le diverse sessioni tramite *menu* contestuali o ricerca diretta (Figura 2.2).

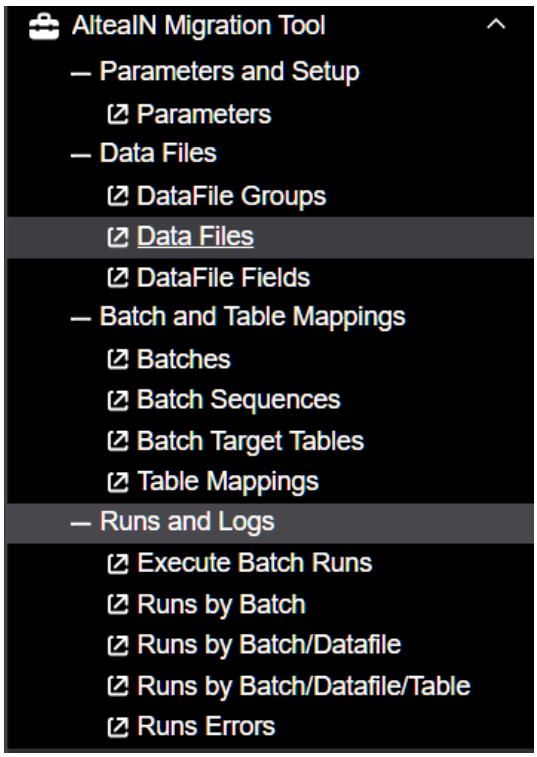


Figura 2.2: Esempio di interfaccia Web Infor LN (LNUI) con elenco sessioni del Migration Tool.

Infor LN è basato su un'architettura che separa il *framework* tecnologico (gli Infor Tools) dalla logica di *business*. Ogni applicazione è composta da diversi componenti:

- **Tabelle (*Tables*)**: I dati fisici, definiti tramite *Data Dictionary*, con domini standard o *custom*.
- **Sessioni (*Sessions*)**: Le interfacce utente (da qui definite come Sessioni) attraverso cui gli operatori accedono, visualizzano e interagiscono con i dati.
- **DAL (*Data Access Layer*)**: L'elemento che si occupa di garantire l'integrità dei dati e delle tabelle, indipendentemente dalle Sessioni [5].
- **Domini (*Domains*)**: Definiscono i tipi di dati, con regole di validazione e formati, che possono essere standard o personalizzati (*custom*).
- **Etichette (*Labels*)**: Utilizzate per la localizzazione e la visualizzazione dei campi nelle Sessioni, con supporto multilingue (es. 02 corrisponde alla lingua inglese, e 06 alla lingua italiano).

2.2.1 Linguaggio Baan C: 3GL e 4GL

Per lo sviluppo e personalizzazione degli *script* e librerie, Infor LN utilizza un linguaggio proprietario, di nome **Baan C** [4].

Esso si divide a sua volta in:

- **4GL (*Fourth-Generation Language*)**: La versione più recente e moderna. Infor LN Tools genera automaticamente operazioni standard di Sessione e database (CRUD). In 4GL non è necessario scrivere codice per gestire il salvataggio o il caricamento di un record, ma ci si concentra sulla gestione degli *eventi* legati ai componenti della Sessione o del database (tramite *DAL* [6]).
- **3GL (*Third-Generation Language*)**: Precedente a 4GL, e totalmente integrato con esso, viene utilizzato per la gestione di logiche di *business* complesse. Permette un approccio più procedurale e a basso livello per implementare funzioni matematiche, algoritmi personalizzati, *DLL* (*Dynamic*

Link Libraries) proprietarie, funzioni e routine di manipolazione stringhe o file, nonché di estensioni (*extensions*) che rappresentano in alcuni casi, l'unico modo di interagire e personalizzare tabelle e procedure standard.

2.2.2 Infor Exchange e Strumenti di Import/Export

Per lo scambio di informazioni, Infor fornisce un modulo denominato *Exchange* [1]. Si tratta di una piattaforma per l'importazione e l'esportazione di file (es. CSV, XML, testo a larghezza fissa e ASCII) mappando righe e colonne sui *Datafiles* e sui campi del database di LN, esso si integra anche con sistemi esterni, permettendone lo scambio di informazioni con Infor.

Spesso sono necessarie delle personalizzazioni delle procedure di migrazione tramite Baan C, che quindi risulta essenziale quando le regole di mappatura superano la complessità gestibile nativamente dall'*Exchange*. Inoltre, un limite architetturale rilevante dell'*Exchange* Infor risiede nella sua inefficienza durante l'elaborazione di grandi quantità di dati: esso tende infatti a elaborare man mano tanti piccoli singoli file. Questo costringe il sistema a ripetere continuamente iterazioni e operazioni di I/O (apertura e chiusura) del tutto evitabili, nonché controlli superflui, le quali impattano pesantemente le prestazioni complessive.

2.3 Migrazione Dati nei Sistemi ERP

2.3.1 Il Paradigma ETL (Extract, Transform, Load)

Per capire appieno le dinamiche e le difficoltà che nascono dal processo di migrazione dei dati, è essenziale inquadrare tali operazioni all'interno del paradigma architetturale *ETL* (*Extract, Transform, Load*) [7]. Questo modello teorico descrive le tre fasi fondamentali necessarie per prelevare dati da uno o più sistemi sorgenti (*source*), spesso eterogenei e obsoleti, per prepararli e infine inserirli in un sistema di destinazione (*target*).

Nello specifico, le tre fasi si articolano nel seguente modo:

- **Extract (Estrazione)**: Rappresenta il processo di lettura e acquisizione delle informazioni dal sistema *legacy*. In questa fase i dati spesso si presentano in formati non strutturati o debolmente strutturati (come file

CSV, esportazioni ASCII a larghezza fissa o vecchi archivi relazionali non normalizzati). La priorità in questa fase è estrarre il dato senza alterarne la semantica originale, gestendo le problematiche di codifica dei caratteri (ad esempio *UTF-8* contro vecchie codifiche *ISO*) e di formattazione.

- **Transform (Trasformazione)**: È la fase più complessa e importante dell'intero paradigma. I dati grezzi appena estratti vengono analizzati, ripuliti e manipolati per adattarsi alle regole di *business* del nuovo *ERP*. In questa fase trovano fondamento i concetti teorici di **Data Cleansing** (pulizia dei dati) e **Data Quality** (qualità del dato). I sistemi *legacy*, spesso in uso da decenni, accumulano tipicamente dati "sporchi" a causa di diverse ragioni: la mancanza di vincoli di integrità referenziale (*Foreign Key*) nei vecchi database relazionali, l'assenza di controlli lato interfaccia utente che consentivano l'inserimento di testo libero al posto di valori con specifici tipi di dominio, l'utilizzo di codifiche obsolete (o stratificate nel tempo a seguito di acquisizioni aziendali) e l'inserimento di dati duplicati. Il processo di trasformazione deve gestire e risolvere queste anomalie strutturali, applicando regole di normalizzazione (ad esempio, imponendo il *casing* corretto tramite funzioni come `toupper` o `tolower`), validando campi obbligatori, decodificando vecchi valori verso i nuovi domini Infor LN, e arricchendo i record con logiche predefinite per garantire che nel nuovo sistema confluiscano esclusivamente informazioni consistenti.

- **Load (Caricamento)**: L'ultima fase consiste nella scrittura fisica dei dati processati nel database *target*. In ambienti complessi, questa scrittura non avviene quasi mai tramite interrogazioni SQL dirette, bensì passando attraverso *API* o livelli di astrazione (come il DAL) che si occupano di avviare eventi aziendali, ricalcolare saldi e far scattare controlli di integrità referenziale.

Lo sviluppo di uno strumento software automatizzato (*tool*), come quello trattato in questa tesi, risponde all'esigenza di coprire tutte le fasi del paradigma *ETL* all'interno del medesimo ecosistema. L'obiettivo è superare le classiche frammentazioni dove sviluppatori diversi usano codici, linguaggi, e metodologie diverse per le tre fasi, aumentando vertiginosamente la probabilità di errori semantici durante il trasferimento delle informazioni.

2.3.2 Sfide e Problematiche Comuni

La migrazione da un sistema *legacy* a un nuovo *ERP cloud* richiede un'attenta fase di analisi per evitare perdite o disallineamenti di informazioni storiche e *master data* [3].

Attualmente, molte migrazioni sono gestite manualmente, con l'integrazioni di *script* scritti *ad hoc*, con un alto livello di complessità e rischio d'errore. Risulta quindi una buona idea la creazione del Tool di Migrazione (*Migration Tool*).

2.3.3 Necessità di Strumenti di Automazione e Generazione Dinamica

Per la migrazione verso il *Cloud*, non è più sostenibile un approccio basato esclusivamente su procedure di inserimento manuali o sullo sviluppo isolato di *script* monouso scritti *ad hoc* per ogni singola tabella aziendale. Questa pratica, oltre a comportare un ingente dispendio di ore lavorative per i consulenti, espone il progetto a continui errori umani derivanti dal copia-incolla delle logiche.

Inoltre, i moderni *ERP SaaS* richiedono strumenti robusti in grado di interfacciarsi con il database non solo in esecuzione *batch* locale, ma anche attraverso servizi *web* (*WebServices* o *API REST*) in scenari distribuiti. Il dato deve essere normalizzato, validato a livello applicativo e controllato prima ancora di essere introdotto al *database server*.

Emerge quindi una lacuna tra i vecchi processi operativi e le stringenti necessità dell'infrastruttura Infor *CloudSuite* e diventa necessario lo sviluppo di *framework* di migrazione che automatizzino la fase di scrittura del codice: partendo da mappature logiche definite dall'utente tramite interfacce grafiche (per quanto possibile), il *tool* deve essere in grado di generare a *runtime* le procedure di conversione, con un'efficienza migliore dell'*Exchange Infor*. Solo eliminando la scrittura manuale degli script di mapping è possibile garantire la ripetibilità del processo, una diagnostica precisa degli errori e la totale tracciabilità di ogni singolo *record* rifiutato.

2.4 Limiti delle Tecnologie Alternative

Sebbene si fosse valutato sul piano puramente teorico l'utilizzo dei *Web Services* SOAP (o anche a un aggiornamento ai *Web Services REST* sperimentali) messi a disposizione da Infor LN (tramite *Infor ION* o BODs), essi sono stati scartati in quanto incompleti, spesso datati rispetto agli ultimi moduli applicativi, e fondamentalmente inadatti alle *performance* richieste e alla complessità di manipolazione di grandi moli di dati (mancanza di primitive complete per elaborazioni *batch* complesse).

Inoltre, molti *endpoint* espongono solo un sottoinsieme minimale di funzionalità o risultano internamente protetti da Infor stessa e resi quindi inutilizzabili per sviluppatori esterni e consulenti ai fini della migrazione dati. Lo sviluppo nativo *backend* tramite Baan C è risultato quindi l'unica soluzione adatta, *performante* e completamente integrabile con le rigide architetture dell'ERP. Si è quindi scelto di sviluppare in ambiente nativo anziché tramite integrazione, magari apparentemente più moderna, con gli strumenti esterni analizzati precedentemente.

Capitolo 3

Analisi dei Requisiti

3.1 Scenario di Riferimento

3.1.1 Necessità di Ottimizzazione

Come evidenziato nei capitoli precedenti, i cambiamenti architetturali da parte di Infor e i nuovi *standard* più restrittivi legati agli ambienti *multitenant* hanno reso le operazioni manuali e l'uso dell'*Exchange Infor* molto più complesse [15]. Emerge quindi la necessità aziendale di innovare e automatizzare il processo con l'introduzione di un nuovo *Migration Tool*.

3.2 Requisiti Funzionali

I requisiti funzionali definiscono i comportamenti attesi, ossia che cosa dovrebbe fare lo strumento, così come stabilito con gli *stakeholder*. Per la classificazione di questi requisiti è stata adottata la metodologia **MoSCoW** (*Must have, Should have, Could have, Won't have*), che permette di definire in maniera chiara le priorità di sviluppo (Tabella 3.1).

Priorità	Descrizione Requisito
Must Have	- Sviluppo dell'*Importer* per i file legacy. - Generatore dinamico di codice Baan C 4GL e 3GL. - Validazione formale del dictionary (DAL). - Gestione massiva (Batch) delle elaborazioni.
Should Have	- Estensibilità con funzioni ad-hoc (*Callback*). - Esportazione documentale integrata con strumenti aziendali preesistenti (*Reports*).
Could Have	- Suggerimento intelligente (o logica basata su *Machine Learning* testuale) per il consiglio automatico dei *mapping Target-Source*.
Won't Have	- Elaborazione distribuita su più thread (non supportata). - Interfaccia grafica dedicata e separata dalla LNUI o BECS.

Tabella 3.1: Classificazione dei requisiti funzionali secondo la matrice MoSCoW.

3.2.1 Sviluppo dell'Importer per Infor Exchange

Il sistema dovrà supportare l'*input* di file e strutture generate dallo strumento preesistente nativo di Infor *Exchange*.

Tali file presentano specifici *layout* (come formati testuali *ASCII*) che dovranno essere interpretati per popolare il dizionario `txamg110` (*Datafiles*) e `txamg111` (*DataFile Fields*). A differenza dell'approccio frammentato dell'*Exchange* di Infor, adotteremo una soluzione ottimizzata: il *tool* processerà l'intero flusso di metadati a partire da un singolo file sorgente, limitando e rimuovendo del tutto l'elaborazione a piccoli blocchi su singoli file che rallentano l'esecuzione iniziale dell'*import*. In altre parole, il file sorgente sarà un singolo file generato dall'importer, contenente tutte le tabelle e dati dalla fonte legacy da cui si sta migrando, e il *tool* farà un controllo completo iniziale, e passerà poi all'elaborazione in *batch* dei *runs*.

Internamente al *Migration Tool* sarà sviluppato e integrato uno strumento generatore di codice. Partendo dalle tabelle di *mapping* (es. `txamg203`), lo strumento dovrà generare *script* di elaborazione e conversione, senza l'intervento manuale del programmatore.

3.2.2 Estensibilità con Funzioni Ad-hoc

Nonostante l'automazione, alcune regole *legacy* di *business* non potranno essere legate a semplici assegnazioni.

Viene quindi la necessità di una funzionalità di *callback* per permettere, ad opera dello sviluppatore, l'inserimento di *script* personalizzati che verranno inseriti nel processo principale durante l'esecuzione *batch* (`txamg3200`).

3.2.3 Integrazione con lo Strumento Documentale

Per garantire il raggiungimento degli *standard* del progetto, il *Migration Tool* dovrà interfacciarsi con la documentazione aziendale, permettendo la produzione ed esportazione delle specifiche tecniche di migrazione, *mappature*, eccezioni e *log* generati in fase di esecuzione (necessari per *debugging*).

3.3 Requisiti Non Funzionali

I requisiti non funzionali gestiscono i vincoli qualitativi e infrastrutturali del software[8].

3.3.1 Manutenibilità ed Estensibilità

Il *Migration Tool* dovrà essere interamente sviluppato in linguaggio Baan C rispettando il *framework* Infor Tools, così da garantirne la compatibilità con future *release* dell'*ERP* e manutenibilità semplificata per il comparto IT aziendale. Tale scelta rappresenta inoltre uno stringente vincolo architetturale: l'ecosistema Infor LN non permette agevolmente altre alternative per questo genere di operazioni `massive`. Come anticipato nel precedente capitolo, lo sviluppo nativo *backend* tramite Baan C è risulta quindi l'unica soluzione adatta, *performante* e completamente integrabile.

3.3.2 Sicurezza e Data Integrity in Multitenancy

Dato che il sistema tratterà dati di tipo *core* (come informazioni private o riservate) dell'azienda cliente in contesti *Cloud*, la sicurezza e la segregazione (isolamento) dei dati per i *Tenant* costituiscono un requisito fondamentale.

Inoltre, il caricamento dovrà ottimizzare l'uso della memoria e delle sessioni, prediligendo un'elaborazione a blocchi al fine di minimizzare i rallentamenti causati dalla latenza di rete verso i server *CloudSuite* remoti.

3.3.3 Usabilità (UX/UI)

Dato che il *tool* sarà utilizzato da consulenti applicativi (quindi non necessariamente programmatori), l'interfaccia utente dovrà rispettare fedelmente le stringenti linee guida di *Infor LN User Experience*. In particolare, per prevenire errori umani durante le fasi di *Data Entry* e mappatura, il sistema dovrà limitare al minimo i campi a testo libero, sostituendoli ove possibile con componenti di *lookup* o *zoom* obbligatori, che forzano l'utente a selezionare referenze valide (ad esempio, la selezione della tabella *Target* da un elenco precaricato e non tramite inserimento manuale della stringa). Questo paradigma, permette di ridurre drasticamente la probabilità di configurazioni errate che impatterebbero a cascata sulla generazione del codice.

3.4 Casi d'Uso (Use Cases)

Per rappresentare le interazioni tra l'utente (il consulente / sviluppatore) o i sistemi esterni e il *Migration Tool*, è stato realizzato il diagramma dei casi d'uso (Use Case Diagram UML).

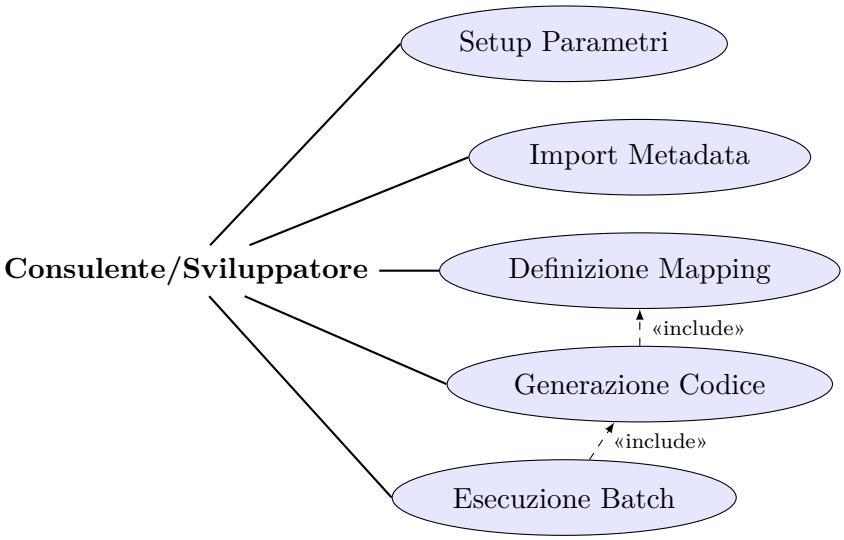


Figura 3.1: Diagramma UML dei Casi d'Uso del Migration Tool.

Come illustrato nel diagramma di flusso e UML (Figura 3.1), per ogni scenario sono state definite le azioni intraprese del sistema. Vengono quindi mostrate di seguito le tabelle descrittive per ciascun caso d'uso del *Migration Tool*.

Caso d'Uso	**Setup Parametri**
Attore Principale	Utente/Sviluppatore
Pre-condizioni	Accesso alla sessione dei parametri generici (`txamg000`).
Flusso Principale	L'utente accede ai parametri e configura i percorsi dei file, i package/moduli di riferimento.
Post-condizioni	I parametri fondamentali sono salvati.
Eccezioni	Errore nel salvataggio, parametri obbligatori mancanti.

Tabella 3.2: Caso d'uso: Setup Parametri.

Caso d'Uso	**Import Metadata**
Attore Principale	Utente/Sviluppatore
Pre-condizioni	I file legacy e le loro definizioni (tracciati record) sono pronti.
Flusso Principale	L'utente richiama la routine *Importer* che interpreta i metadati dal formato sorgente e popola i dizionari (`txamg110`, `txamg111`).
Post-condizioni	Il dizionario *input* auto-compilato con le strutture dati esterne.
Eccezioni	Formato sorgente corrotto, tipo non supportato, mancanza di permessi I/O.

Tabella 3.3: Caso d'uso: Import Metadata.

Caso d'Uso	**Definizione Mapping**
Attore Principale	Utente/Sviluppatore

Continua nella pagina successiva

Caso d'Uso	Definizione Mapping (Continua)
Pre-condizioni	I metadati sorgente sono stati importati e le tabelle LN target sono note.
Flusso Principale	L'attore associa i file d'ingresso alle tabelle Infor (Target Tables `txamg202`) e definisce le regole campo per campo (`txamg203`).
Post-condizioni	Le logiche di *mapping* sono finalizzate nel Data Entry.
Eccezioni	Campi target non esistenti, errata associazione del tipo di dato.

Tabella 3.4: Caso d'uso: Definizione Mapping.

Caso d'Uso	Generazione Codice
Attore Principale	Sviluppatore
Pre-condizioni	Mapping correttamente definito per un determinato Batch.
Flusso Principale	L'utente avvia l'interfaccia di generazione. Il motore converte il database di regole in script nativi Baan C (inclusi array, API DAL, check). Compila il codice in *object-file* eseguibili.
Post-condizioni	Codice generato, validato dal compilatore interno e pronto per il *deploy*.
Eccezioni	Regole personalizzate complesse errate (sintassi non valida), lock sui file da compilare.

Tabella 3.5: Caso d'uso: Generazione Codice.

Caso d'Uso	Esecuzione Batch
Attore Principale	Utente/Sviluppatore
Pre-condizioni	Il codice generato compilato. È stato parametrizzato un *Run* (`txamg300`).

Continua nella pagina successiva

Caso d'Uso	Esecuzione Batch (Continua)
Flusso Principale	L'utente lancia il caricamento, leggendo fisicamente i file dati grezzi e instanziando la validazione ed inserimento via DAL sul server LN.
Post-condizioni	Dati importati nell'ERP. *Log* di risultato salvati nelle specifiche tabelle interne.
Eccezioni	Record in errore per vincoli DB (Foreign Key o Domain constraint), file dati *missing* o disconnessioni server.

Tabella 3.6: Caso d'uso: Esecuzione Batch.

3.5 Modello dei Dati

3.5.1 Analisi Strutture Dati Coinvolte

Per supportare in modo efficace il processo di migrazione, è stata definita un'architettura dati, basata su package *tx* (*custom*).

Il modello dati è suddiviso in diverse entità:

- **Parametrizzazione**: tramite tabelle di configurazione come `txamg000` (AlteaIN Migration Tool Parameters).
- **Gestione dei Datafile**: tabelle come `txamg110` (Datafiles) e `txamg111` (DataFile Fields) permettono di definire le strutture dei file originali da importare.
- **Elaborazione dei Batch**: la configurazione e l'esecuzione tramite batch è regolata da tabelle come `txamg200` (Batch), `txamg201` (Batch Sequences) e `txamg202` (Target Tables), insieme ad altre tabelle dedicate al mapping dei campi (`txamg203`).
- **Tracciamento delle Esecuzioni**: lo stato e l'esito delle operazioni, oltre all'eventuale presenza di errori, vengono memorizzati all'interno delle tabelle `txamg300`, `txamg301`, `txamg302` e `txamg303` (Runs Errors).

I Domini includono sia tipi strettamente descrittivi (es. `txamg.cdom`, `txamg.fdnm`) sia tipi enumerativi per la validazione e il workflow (es. `txamg.migt` o `txamg.mapl`).

Esempi di Domini essenziali sono quelli relativi ai formati ASCII (`txamg.fast`), alle regole di mapping (`txamg.mapl`) o al tracciamento dei file (`txamg.dfil`).

Schema ER del modello dati del Migration Tool: (Figura 3.2)

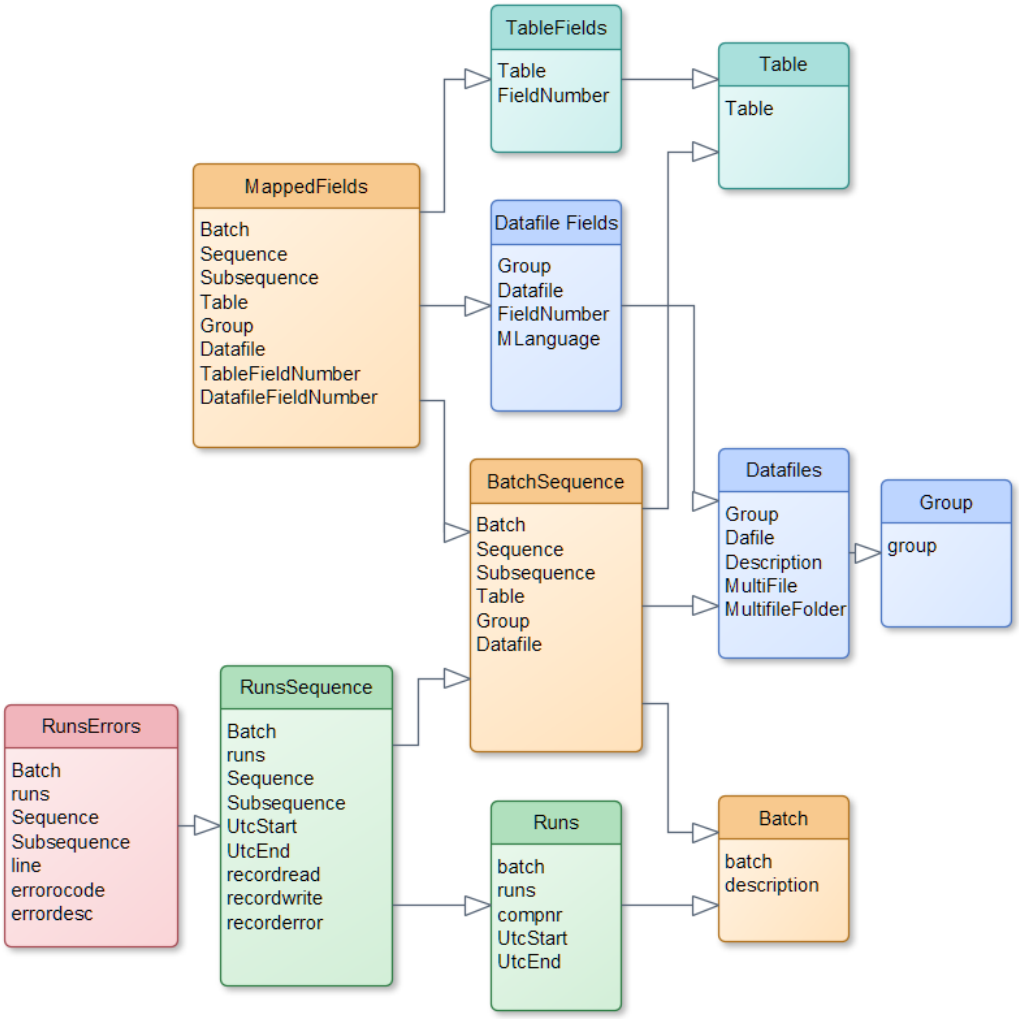


Figura 3.2: Schema ER del Modello Dati del Migration Tool

Tabella delle tabelle principali del Migration Tool:

Tabella	Nome logico	Funzione
txamg000	AlteaIN Migration Tool Parameters	Parametri globali e configurazione generale del tool
txamg100	Table groups	Parametri specifici per i batch di migrazione
txamg110	Datafiles	Definizione dei file di input
txamg111	DataFile Fields	Dettagli dei campi dei file di input
txamg200	Batch	Configurazione processi batch di migrazione
txamg201	Batch Sequences	Sequenze e dipendenze tra batch
txamg202	Target Tables	Definizione delle tabelle target in CloudSuite
txamg203	MappedFields	Mapping logico tra campi di origine e destinazione
txamg300	Runs by Batch	Tracciamento delle esecuzioni dei batch
txamg301	Runs by Batch/Datafile	Tracciamento delle esecuzioni per file specifico
txamg302	Runs by Batch/Datafile/Table	Tracciamento delle esecuzioni per tabella specifica
txamg303	Runs Errors	Dettaglio degli errori riscontrati durante le esecuzioni

Tabella 3.7: Dizionario delle Tabelle Principali del Migration Tool

Questi elementi compongono le fondamenta dell'integrazione per la generazione del codice di migrazione e dell'*Importer*.

Capitolo 4

Architettura

4.1 Panoramica Architettura di Sistema

L'architettura del *Migration Tool* è stata definita per essere modulare e scalabile. Alla base del sistema vi è una netta separazione tra il modulo di *input* (*Importer*), il motore per l'elaborazione dei metadati, e il Generatore di Codice. Questo approccio garantisce la distinzione tra il *layout* dei dati preesistenti e la struttura del database Infor *CloudSuite*.

4.1.1 Modello Dati e Dizionario

L'architettura dati è stata progettata per riflettere e tracciare ogni singolo passaggio del ciclo di vita della migrazione. Il modello relazionale si basa su un raggruppamento logico identificato dai prefissi delle tabelle (sviluppate nel *package custom* `tx` / modulo `amg`). Le strutture dettagliate di tali tabelle sono consultabili nell'Appendice A.

- **Setup e Parametrizzazione (`txamg000`)**: Raccoglie i parametri globali del *tool*, incluse le impostazioni di base per l'I/O (`txamg000.bpat`) e i *package*/moduli su cui generare il codice. L'interfaccia di configurazione (Figura 4.1) permette di definire i parametri globali.

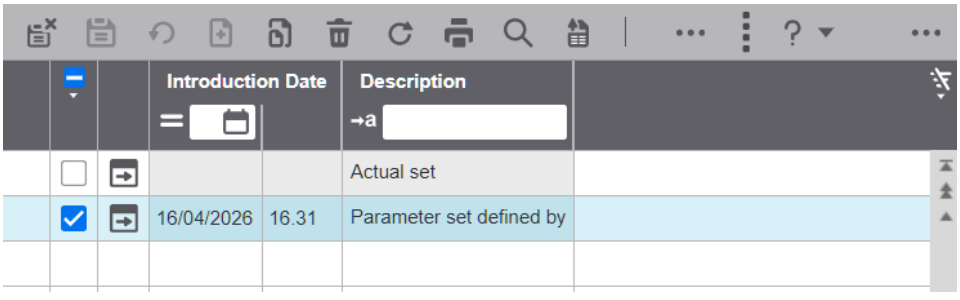


Figura 4.1: Sessione di setup dei parametri globali (`txamg0100m000`).

- **Gestione Metadati (`txamg1xx`)**: Le tabelle `txamg110` (*Datafiles*) e `txamg111` (*Datafile Fields*) costituiscono il dizionario della sorgente. Mappano i file di *input*, includendo il formato (es. separatori *ASCII*, lunghezza fissa) e i domini di ciascun campo (tipo di dato, posizioni iniziali e lunghezze).

- **Regole di Mapping (`txamg2xx`)**: Rappresentano la parte di logica transazionale. In `txamg202` (*Target Tables*) si definiscono le entità, le logiche di aggiornamento del database e i vincoli di condivisione (*Shared*). Successivamente con `txamg203` (*Mapped Fields*), vengono stabilite le relazioni uno-a-uno e uno-a-molti, standardizzate per le funzioni obbligatorie o opzionali.

- **Esecuzione e Log (`txamg3xx`)**: Il sistema adotta una politica di esecuzione strutturata in *run* (elaborazioni). Attraverso `txamg300`, `txamg301` e `txamg302` viene memorizzato il conteggio esatto dei record letti, processati o ignorati in base a tempi e sequenze. Eventuali criticità e troncamenti vengono inseriti sulla tabella `txamg303` (*Run Errors*).

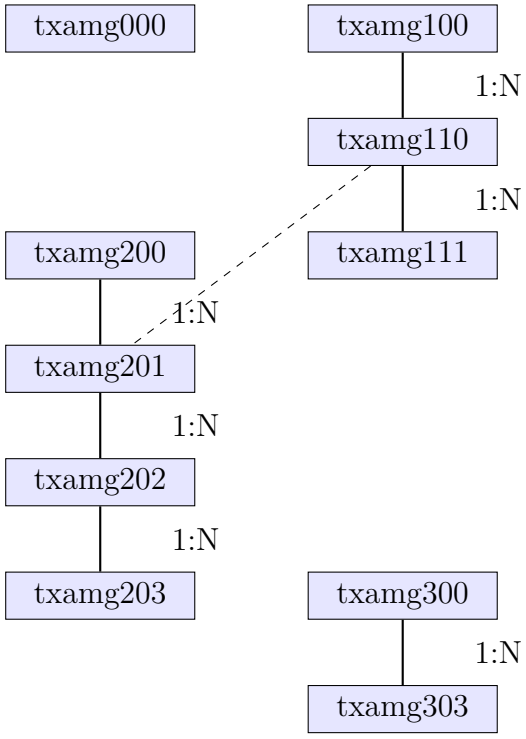


Figura 4.2: Diagramma ER semplificato del *package* txamg.

La Tabella A.1 nell'appendice mostra i parametri di configurazione per gestire i percorsi di I/O e le librerie dinamiche.

4.2 Progettazione del Software

4.2.1 Design dell'Importer e del Parser dei Dati

L'Importer funge da punto d'ingresso per i dati ed è progettato per interpretare il formato proprietario di Infor *Exchange*, analizzando file di esportazione testuali o strutturati. Estraendo i metadati dai record, l'Importer popola i dizionari *Datafiles* (`txamg110`) e i relativi *Fields* (`txamg111`), la cui struttura è definita rispettivamente nelle tabelle A.3 e A.4.

Questo *design* del caricamento rende possibile supportare sia architetture *multitenant cloud*, sia installazioni *legacy on-premise*. L'architettura proposta garantisce una lettura massiva e aggregata da un singolo sorgente aziendale.

4.2.2 Sequenza Operativa del Sistema

Il seguente *Sequence Diagram* ad alto livello illustra il flusso logico tra i componenti logici: l'utente, l'interfaccia (Sessioni Infor), il Motore Core (DLL di generazione) e il Database.

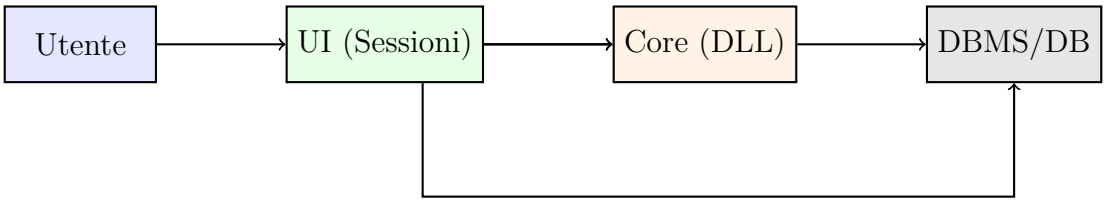


Figura 4.3: Interazione semplificata delle operazioni del sistema.

Per inquadrare microscopicamente l'attività di *Load* vera e propria, è utile osservare il ciclo di vita (State Machine Diagram) di un singolo Record, a partire dalla sua forma grezza su file fino all'inserimento (o scarto).

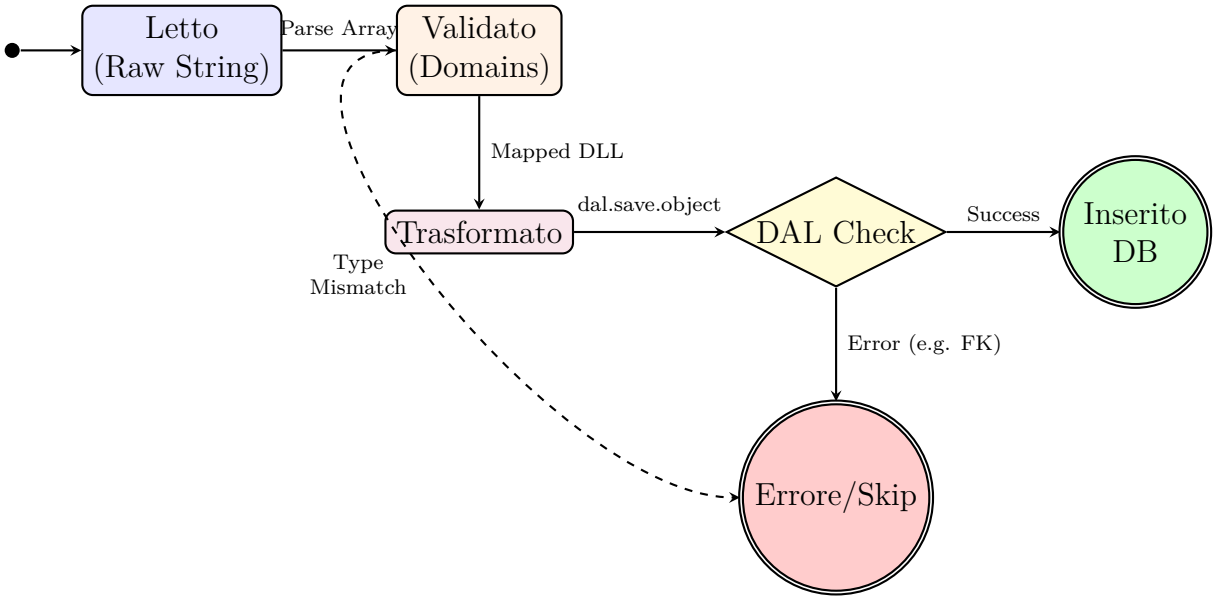


Figura 4.4: Diagramma degli Stati (UML) semplificato di un record sottoposto alla conversione.

4.2.3 Diagramma dei Componenti

L'architettura può essere descritta logicamente attraverso il seguente diagramma a blocchi dei moduli, dove il Motore Generale si divide in *importer* (*parser*), *generator* e *executor*, e comunica coi pacchetti Infor Tools e DAL.

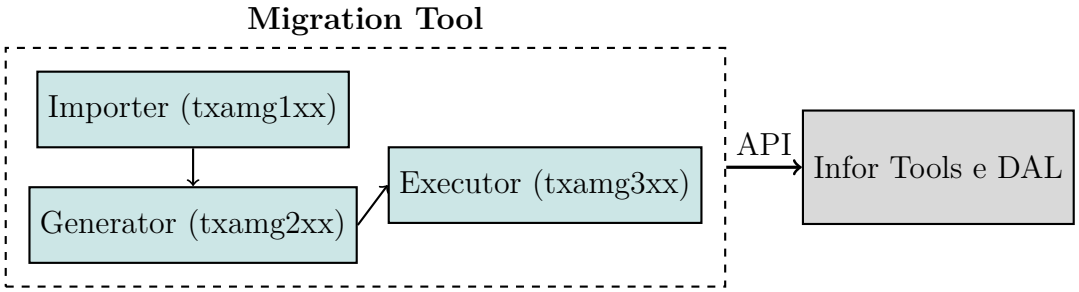


Figura 4.5: Component Diagram dei moduli dell'architettura.

4.2.4 Generazione del Codice

Il generatore di codice è basato su librerie compilate (*DLL* come `txamgdll0012` e `txamgdll0013`), il modulo interpreta il dizionario di *mapping* (definito in `txamg203` (*MappedFields*)) e traduce i legami e le regole di trasformazione in *script* nativi Baan C.

Questo meccanismo previene la scrittura manuale di codice, garantendo il rispetto del *framework CloudSuite*: il codice viene prima generato, poi compilato

via `bic_compiler`, ed infine agganciato ai *Batch Runs*. Al fine di rispettare le rigide *policy* di sicurezza e isolamento dei tenant in ambienti CloudSuite, la compilazione e il deployment delle librerie avvengono esclusivamente all'interno di ambienti di Sviluppo/Staging autorizzati. Successivamente, le DLL validate vengono promosse in Produzione attraverso i canali standard di rilascio dell'ERP, garantendo l'integrità del sistema.

4.2.5 Code Generation vs. Interpretation

Nell'architettura software di questa tesi, in azienda si è discusso se utilizzare un paradigma di esecuzione *interpretata* o optare per una pura *generazione del codice* (*Code Generation*). Quest'ultima tecnica rientra nell'ampio paradigma del **Metaprogramming** (Metaprogrammazione).

In informatica, la metaprogrammazione è la capacità di un programma di trattare altri programmi come propri dati: significa poter leggere, generare, analizzare o trasformare codice sorgente. A livello ingegneristico, esistono diversi approcci per ottenere dinamicamente comportamenti non pre-scritti:

- **Reflection** (es. Java o C#): Analisi a (*runtime*) della struttura di classi e attributi. Sebbene flessibile, richiede pesanti astrazioni da parte della *Virtual Machine* non adatte a un ERP *legacy-oriented*.
- **Interpretazione a runtime**: Un *interpreter* costruito nativamente in Baan C che legge riga per riga il tracciato stringa e applica logiche condizionali riga-per-riga. Questo approccio ha un enorme peso sulle performance (*overhead*) rallentando le elaborazioni sui milioni di record a causa dell'esecuzione sequenziale delle query.
- **Generazione di codice (*Lexical/AST Generation*)**: Trasformazione delle logiche (nel nostro caso i Mapping e Datafile) in *script* nativi (*.bc*) e compilazione prima dell'esecuzione vera e propria. Questo è l'approccio che è stato scelto per questo progetto.

Al contrario del vecchio strumento che offriva interpretazioni particolarmente lente, il *tool* da noi creato utilizza il motore *generatore di codice* compilato. I metadati dell'origine e di destinazione vengono tradotti in script Baan C. Questi script vengono quindi registrati dal server Infor e diventano immutabili

(a meno di un ricalcolo). Solo la *fase preparatoria* (*code generation*) richiede secondi d'elaborazione, permettendo alla successiva fase di inserimento (*Batch*) di essere eseguita alla velocità nativa consentita dal compilatore *Infor*.

4.2.6 Modulo di Integrazione Ad-hoc e Callbacks

Mentre buona parte dei flussi vengono standardizzati automaticamente, alcune regole di *business* richiedono sviluppi specifici a causa della loro complessità. Vengono quindi esposte a livello architetturale delle interfacce di *callback*.

In pratica, in appositi *module script*, gli sviluppatori possono inserire blocchi di codice chiamati asincronamente dal motore generatore di codice e dall'esecutore. Questi *script* personalizzati usufruiscono degli stessi vantaggi transazionali dell'utility principale.

4.3 Scelte Tecnologiche

4.3.1 Linguaggi e Framework

L'intero ecosistema è sviluppato sfruttando il *Framework* Infor Tools, programmando in **Baan C (sia 3GL che 4GL)**. In parte sono state adottate le *DAL (Data Access Layer)*, per la parte restante, sono state utilizzati approcci più classici e a basso livello.

4.3.2 Librerie di Supporto e Strumenti di Build

Tutto il coordinamento della configurazione, *logging* (`txamgdll0011`) e le *string utility* si basano sulle librerie del sistema *ERP* (*DLL* native Baan e *tool* come il `ttdll string`).

Le *build* non prevedono *toolchain* esterni complessi (come *Maven* o *MSBuild*), ma si appoggiano direttamente al *toolset* interno dell'*ERP*, il quale gestisce i legami *VRC* (*Version Release Customer*) *multitenant* per garantirne l'incapsulamento del codice generato al corretto *Tenant*.

Capitolo 5

Implementazione

5.1 Sviluppo dell'Importer Infor Exchange

5.1.1 Gestione dei Formati e Parsing

L'importer è stato sviluppato come un modulo d'*input* che si interpone tra i file d'ingresso (prodotti dai vecchi server o estratti tramite *tool legacy*, a volte anche Access) e le tabelle di definizione del nuovo sistema.

La libreria dedicata alla gestione dei *metadata* (`txamgdll0011`) prende in carico l'interpretazione dei metadati (quali formati delimitati o logiche *ASCII* a lunghezza fissa) eseguendo conversioni su base parametrica secondo l'enumerato `txamg.fast`.

Ogni riga recuperata viene controllata per garantire la conformità delle codifiche testo e i troncamenti (*trim*) o *padding* dei campi vuoti. Durante questa fase il generatore effettua operazioni di *concatenazione stringhe* (utilizzando funzioni come `concat` e `string.scan`). L'implementazione di un sistema con concatenazione di stringhe in linguaggi *legacy*/Baan C richiede particolare attenzione poiché le stringhe spesso non hanno dimensionamenti fissi. Tuttavia, un vantaggio della manipolazione nativa e deterministica con domini è la limitazione dei *Buffer Overflow* e l'ottimizzazione dell'utilizzo della *RAM*, ma d'altro canto, sono necessari continui `offset-trimming` e calcoli (come la lunghezza in base all'*ASCII subset*).

L'*importer* esegue il *parsing* del flusso estrapolandolo da un singolo file sorgente (o su limitati file macro per l'intero transazionale). Questo design permette un *overhead* minore per ciascuna interazione.

```
function void GeneraImportUnpack(domain txamg.dfil datafile)
{
    |* Generazione dinamica della scansione della riga ASCII
    |* print_format conterrà una stringa tipo "|%s|%s|%s|"
    |* string_list conterrà la lista delle variabili di destinazione

    select txamg111.* from txamg111
    where txamg111._index1 = {:txamg110.grou, :txamg110.dfil}
    order by txamg111.fdno
    selectdo
        print_format = print_format & first.pipe & "%s|"
        string_list = string_list & comma & "imp.str."
                      & trim$(txamg111.fdnm)
        comma = ","
        first.pipe = ""
    endselect

    Riga("function extern long unpack."&datafile&"(...) {")
    Riga("    ret = string.scan(buffer_"
              &trim$(datafile)&","""
              &trim$(print_format)&""","&trim$(string_list)&")")
    Riga("    ...")
    Riga("}")
}
```

Listing 1: Esempio di generazione del codice per il *parsing* di stringhe *ASCII* (da `txamgdll0012`).

In questo estratto viene mostrato come il generatore iteri sui campi definiti per il *datafile* (`txamg111`) per costruire una chiamata a `string.scan`. Questa funzione Baan C viene poi usata nello *script* generato per «spacchettare» la riga letta dal file *ASCII* nelle variabili corrispondenti ai singoli campi.

5.1.2 Ricostruzione delle Definizioni dei File

L'obiettivo secondario dell'*importer* è auto-compilare il *setup* dello stesso *Migration Tool*. Utilizzando le definizioni originarie dei file (le tabelle *legacy*), il programma crea dinamicamente e aggiorna una mappatura interna dei *Data Files* attraverso Sessioni come `txamg1110m000`.

Le descrizioni, numerazioni dei campi (`txamg.fdno`) e i tipi di dominio coinvolti (`txamg.atyp`) vengono inseriti nella tabella di supporto `txamg111` e sono quindi disponibili e precompilati per le successive attività di *data-mapping* manuale verso le tabelle di destinazione della *CloudSuite*.

L'utente può gestire singoli file sorgente e gruppi di essi tramite le sessioni mostrate in Figura 5.1 e 5.2.

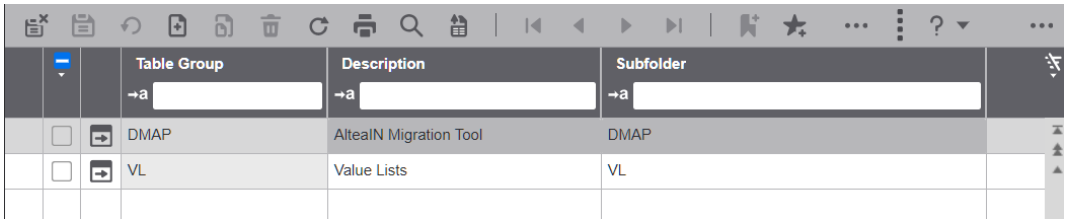


Figura 5.1: DataFile Groups (`txamg1100m000`).

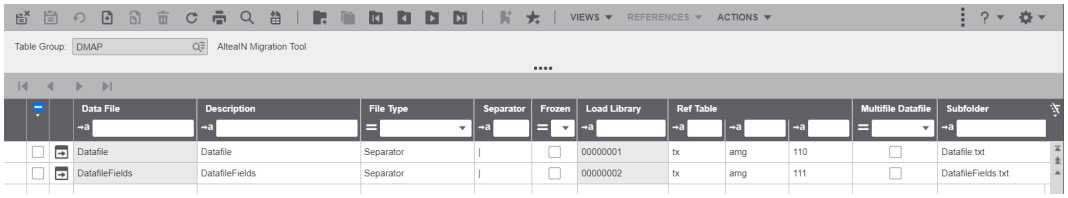


Figura 5.2: DataFiles (`txamg1110m000`).

I dettagli dei campi per ogni file, inclusi i domini e le lunghezze, è gestito in una sessione specifica (Figura 5.3).

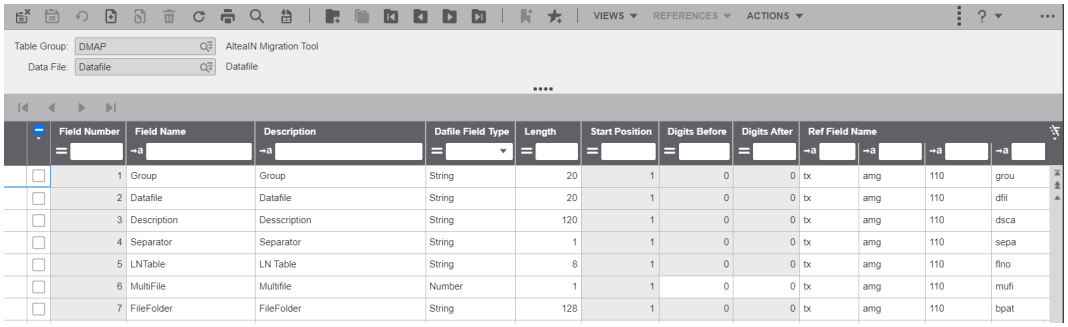


Figura 5.3: Datafile Fields (`txamg1111m000`).

5.2 Implementazione del Generatore di Codice

5.2.1 Creazione Dinamica del Codice

La generazione dinamica del codice è gestita mediante apposite librerie *DLL* (*Dynamic Link Library*) create all'interno del sistema Infor, quali `txamgdll0012` (*Script Generator - Load dll*) e `txamgdll0013` (*Script Generator - Process dll*).

Il codice sorgente Baan C viene generato riga per riga, definendo le firme delle funzioni (es. `function extern long process.import.file...()`) e le variabili necessarie. Per farlo in modo conforme all'infrastruttura di Infor LN, il generatore interroga direttamente il *Data Dictionary* del sistema tramite le tabelle `ttadv` (in particolare `tttadv420` per le definizioni delle tabelle, `tttadv422` per i campi e `tttadv400` per i domini). Questa interrogazione viene successivamente delegata al componente `txamgdll0010` (*DD Integration Library*), che funge da strato di astrazione estraendo in automatico i tipi di dato, lunghezze massime e *array* per assicurare che lo *script* rispetti le definizioni fisiche corrette.

Tali librerie servono ad analizzare la configurazione definita nelle tabelle di *mapping* (ad esempio `txamg203`) per generare script eseguibili, riducendo il tempo manuale richiesto per configurare le regole di conversione standard e garantendo che il codice finale aderisca alla logica di business (*business logic*).

```
1  function void GeneraMap(domain tcmcs.str8 tabella)
2  {
3      Riga("function long map."&tabella&".record() {")
4      select txamg203.*, txamg111.* from txamg203, txamg111
5      where txamg203._index1 =
6      {:txamg202.cbat, :txamg202.sequ, :txamg202.sseq}
7      and txamg203.cmbd refers to txamg111
8      selectdo
9          on case txamg203.type
10             case txamg.type.long:
11                 Riga("    "&tabella&trim$(txamg203.fdnm)
12                          &" = lval(imp.str."&trim$(txamg203.fdnm)&")")
13                 break
14             case txamg.type.string:
15                 Riga("    tt.align.according.domain(imp.str."
16                         &trim$(txamg111.fdnm)&","
17                         &tabella&trim$(txamg203.fdnm)&",...)")
18                 break
19             |* Altri tipi (date, enum, etc.)
20         endcase
21     endselect
22     Riga("    return(OK)")
23     Riga("}")
24 }
```

Listing 2: Logica generazione della funzione di mapping (da `txamgdll0013`).

Nell'esempio viene mostrata la funzione `GeneraMap` che genera il codice per mappare i dati estratti dal file d'origine alla struttura della tabella di destinazione:

- **Riga 4-6**: Esegue un ciclo sui campi mappati (`txamg203`) relazionandoli con i campi sorgente (`txamg111`).
- **Riga 9-10**: Se il target è un numero intero (*long*), genera una chiamata a `lval()` per convertire la stringa ASCII in valore numerico.
- **Riga 12-13**: Se il target è una stringa, usa `tt.align.according.domain()` per gestire lunghezze e allineamenti secondo il dominio Infor LN.

5.2.2 Funzioni di Trasformazione Ad-hoc

Oltre alle regole predefinite, l'interprete permette la dichiarazione e l'inserimento di funzioni personalizzate create ad-hoc. Durante il processo, eventuali routine di validazione o pulizia del dato possono essere inoltrate ad altre interfacce di esecuzione (come `txamg3200` - *Execute Batch Runs*).

Bisogna quindi consentire allo sviluppatore di interagire con il tool attraverso *callback* specifici. Per esempio, nel caso in cui una data proveniente dal sistema *legacy* necessiti di *parsing* complesso o validazioni logiche *custom*, lo sviluppatore può definire uno script aggiuntivo inserito in fase di *mapping*. Ecco un esempio teorico pratico del codice di una funzione di trasformazione implementata in Baan C:

```
function extern domain tcdate
converti.data.legacy(domain tcmcs.str10 legacy_date)
{
    long yy, mm, dd
    domain tcdate target_date

    |* Parsing manuale "YYYY-MM-DD" o formati non standard
    |* Questo if serve a verificare se sia vuota o nulla
    if isspace(legacy_date) then
        |* Return con data nulla
        return(0)
    endif

    yy = lval(legacy_date(1;4))
    mm = lval(legacy_date(6;2))
    dd = lval(legacy_date(9;2))

    |* Validazione e conversione nel formato numerico (giorni) di LN
    if mm >= 1 and mm <= 12 and dd >= 1 and dd <= 31 then
        target_date = date.to.num(yy, mm, dd)
        return(target_date)
    else
        dal.set.error.message("@Data non valida legacy: %s", legacy_date)
        return(0)
    endif
}
```

Listing 3: Esempio di funzione di trasformazione *ad-hoc* sviluppata per manipolare date non standard.

Il componente `txamgdll0011` si occupa del management dei metadati e dei log. Inoltre, queste *DLL* scambiano informazioni a *runtime* mediante passaggio di parametri rigorosamente definiti o variabili globali *extern* accessibili alla sessione in esecuzione. Il collegamento tra DLL proprietarie e codice generato si basa su puntatori allocati temporaneamente, essenziali in Baan C per l'invio di costrutti complessi senza pesare sul *Data Access Layer* sottostante.

5.2.3 Gestione delle Latenze e Fallback Dinamico

Un aspetto dell'implementazione riguarda la **gestione delle eccezioni** legate alle latenze di rete e all'integrità referenziale, specialmente importante trattan-

dosi di un inserimento massivo in ambiente Cloud (*CloudSuite*). Per superare le inefficienze della singola *commit*, è stato adottato un approccio *Array-Insert* (inserimento a blocchi di centinaia di record alla volta tramite *array*). Tuttavia, qualora la DAL (*Data Access Layer*) riscontri un errore formale (*Foreign Key* mancante, *Domain Check* fallito) in uno solo dei record contenuti nell'*array*, Infor scarterebbe l'intera transazione annullando di conseguente l'intero blocco e inserendolo nella lista di *rollback*.

Per gestire questa criticità, è stata integrata una `tecnica di fallback`: al fallimento del blocco, il sistema entra in un ciclo di *retry* che esegue automaticamente la scansione e l'inserimento dello specifico blocco di dati riga per riga. Tale meccanismo garantisce che il dato *sanitizzato* venga salvato e che solamente la riga errata venga scartata, fornendo *log* più precisi e massimizzando al contempo le prestazioni dell'inserimento.

```
function long ProcessaBlocco(domain tcemm.noma num.righe)
{
    long obj.ret, i

    |* Tentativo di inserimento Array-Insert
    db.retry.point()
    obj.ret = dal.save.object("tccom001", num.righe)
    if obj.ret = 0 then
        commit.transaction()
        return(OK)
    endif

    |* Fallback Dinamico, fallimento inserimento a blocchi. Elaborazione riga per riga
    abort.transaction()
    for i = 1 to num.righe
        db.retry.point()
        obj.ret = dal.save.object("tccom001", 1) |* Inserisco la riga
        if obj.ret = 0 then
            commit.transaction()
        else
            |* Catturo l'errore generato dalla DAL e lo aggiungo ai log
            LoggaErrore(i, dal.set.error.message())
            abort.transaction()
        endif
    endfor
    return(OK)
}
```

Listing 4: Pseudo-codice Baan C che mostra la logica iterativa di transazione e fallback riga-per-riga.

5.2.4 Logging degli Errori

Il processo di validazione e selezione dei record è strettamente legato a una complessa infrastruttura di *logging*. Invece di disperdere i *warning* e gli *errors* nei log dei server di Infor, che non facilmente consultabili, le stringhe non accettate di ciascuna tabella vengono salvate, attivando il `dal.set.error.message` e scritte nella tabella `txamg303` (*Runs Errors*).

Un esempio di logica di recupero e salvataggio dell'errore (chiamata dalla funzione `LoggaErrore`) è la seguente:

```
function long LoggaErrore(long riga.id, string str.messaggio)
{
    |* Chiave tabella txamg303
    txamg303.cbat = current.batch
    txamg303.dfil = current.datafile
    txamg303.reno = riga.id

    |* Inserimento messaggio formattato
    txamg303.mesg = str.messaggio
    db.insert(ttxamg303, db.retry)
    commit.transaction()
    return(OK)
}
```

Listing 5: Snippet dimostrativo di inserimento del log di errore scaturito dal livello DAL interno.

Tale implementazione è utile per l'utilizzatore finale in quanto avrà visibilità del log testuale completo dalla LNUI, tramite le sessioni di logs come quella degli errori (Error Logging).

5.2.5 Gestione delle Latenze Transazionali

Nel *Tool* è stato implementato un sistema personalizzato per l'elaborazione dei *commit* a blocchi durante la trasformazione dati. In ecosistemi *Cloud* da remoto, le singole *transactions* subiscono notevoli rallentamenti dati della latenza causate dal *round-trip* di ogni operazione di *INSERT* o *UPDATE* verso i *layer* esterni di Infor in *datacenter* remoti. Raggruppando migliaia di record (*array-insert*), come detto precedentemente, l'impatto della latenza diminuisce (fino al 90%). Di contro, l'impiego delle transazioni a gruppi necessità di particolare attenzione per la gestione del *rollback*, parziale o completo, di blocchi nel caso che anche solo un inserimento si corrompa (o violi i controlli della *DAL*). Per mitigare questa criticità, il sistema implementa una strategia di *fallback*: in caso di fallimento di una transazione a blocchi, viene catturata l'eccezione e rieseguito l'inserimento per i *record* di quel blocco singolarmente (riga per riga *row-by-row*). Questo approccio permette di isolare il *record* anomalo, eseguendo la *commit* per tutti i restanti dati validi senza interromperne l'intero flusso o *run* migliorando internamente la flessibilità e le prestazioni senza perdita di dati.

5.3 Interfacce Utente (Sessioni)

L'interazione con l'utente avviene attraverso un gruppo di sessioni dedicate, denominate *m-sessions* o *s-sessions* a seconda della loro finalità operativa.

5.3.1 Design dell'Interfaccia Utente e Scelte UX/UI

Lo sviluppo e l'elaborazione dell'interfaccia si sono dovuti adattare a specifici vincoli e *design pattern* di Infor LN, in modo da fornire all'utente una *UX* (*User Experience*) coerente e uniforme.

Gli elementi chiave della *UI* implementati comprendono:

- **Sessioni Multi-Main (Multi-Occorrenza) vs Details (Dettaglio)**: Gran parte delle maschere previste (come l'elenco dei *Datafiles* o *Batch*) sono state progettate come sessioni multi-occorrenza (lista tabellare). L'utente può scorrere e filtrare grandi quantità di *record* e aggiungere o modificare dati *inline*, o richiamare una sessione di dettaglio secondaria in caso di presenza di decine di campi *settings* o a scelta multipla. Questo *pattern* aumenta l'intuibilità e la facilità d'uso dello strumento senza commettere errori, garantendo il rispetto degli standard sull'usabilità LN.
- **Formati e Maschere (Formats)**: I campi a *video* non sono semplici *text-input*, ma si basano su *formati* specifici imposti a livello di *Data Dictionary*. Questa scelta fa in modo che il cursore e il *parsing* dell'input costringano *out-of-the-box* l'inserimento solo di dati previsti e corretti (per esempio: forzando il Maiuscolo, o aggiungendo i separatori alle cifre decimali), eliminando una buona porzione di possibili errori prima ancora dell'esecuzione e successivo salvataggio.
- **Zoom (Lookups/Selettori Dati)**: Altro strumento essenziale nell'interfaccia del *Migration Tool* è l'uso degli *zoom*. Gli *zoom* consentono all'utente, all'interno dei campi di *foreign key* (ad esempio: per scegliere a quale *Package* logico appartiene una tabella, ma non solo), di lanciare una sub-sessione modale per ricercare e prelevare un valore da tabelle o da enum (opzioni predefinite) preesistenti, invece di memorizzare le codifiche mnemonicamente.

Le sessioni principali per la gestione dei `batch` e dei `mapping` sono visualizzabili nelle Figure 5.4, 5.5, 5.6, 5.7.

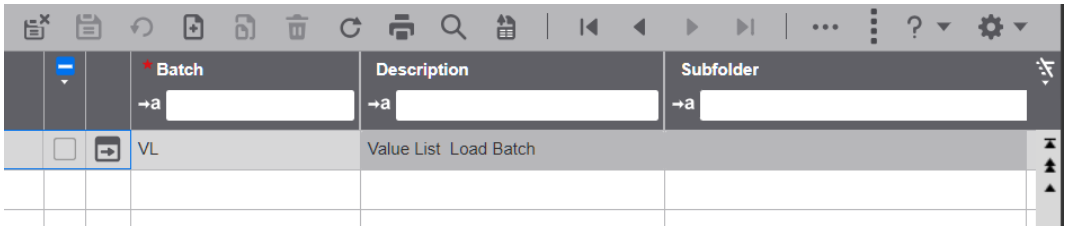


Figura 5.4: Batches (`txamg2100m000`).

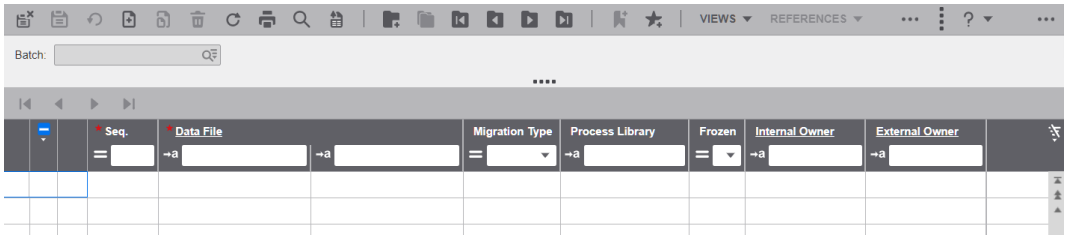


Figura 5.5: Batch Sequences (`txamg2101m000`).

Nelle sessioni dei mapping si definiscono le tabelle target e le logiche di conversione per singolo campo.

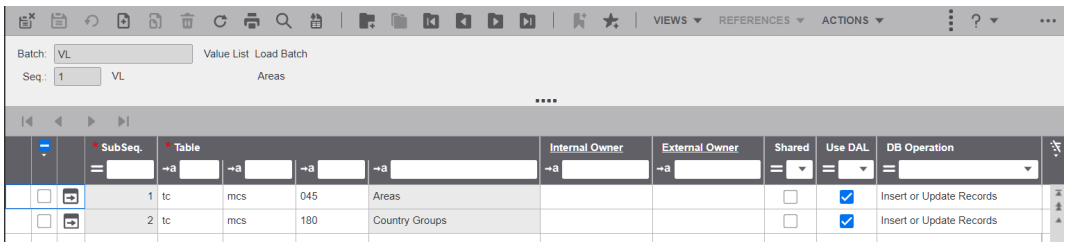


Figura 5.6: Batch Target Tables (`txamg2102m000`).

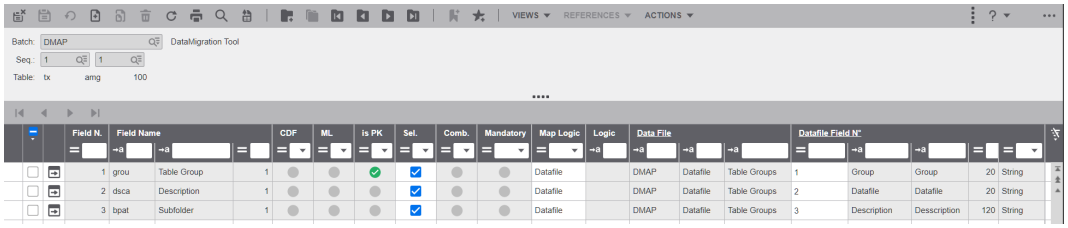


Figura 5.7: Table Mappings (`txamg2103m000`).

Si evidenziano, per esempio:

- **Setup iniziale**: la Sessione `txamg0100m000` permette la parametrizzazione globale dello strumento.

- **Definizione Formati**: le Sessioni relative a `txamg1110` gestiscono il popolamento dei formati dati elaborati.
- **Mapping Dati**: la definizione di conversione logica e associazioni campo-campo tra origine e destinazione avviene nelle interfacce di visualizzazione delle tabelle *target* (`txamg2102m000`) e *fields mapping* (`txamg2103m000`).

Nelle seguenti sezioni vengono riportati gli elenchi dei componenti software sviluppati, suddivisi per tipologia.

5.3.2 Esecuzione e Monitoraggio

Una volta configurati i mapping e generato il codice, il tool permette l'esecuzione dei batch tramite la sessione `txamg3200m000` (Figura 5.8).

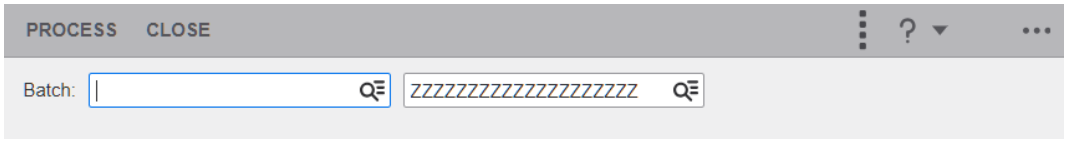
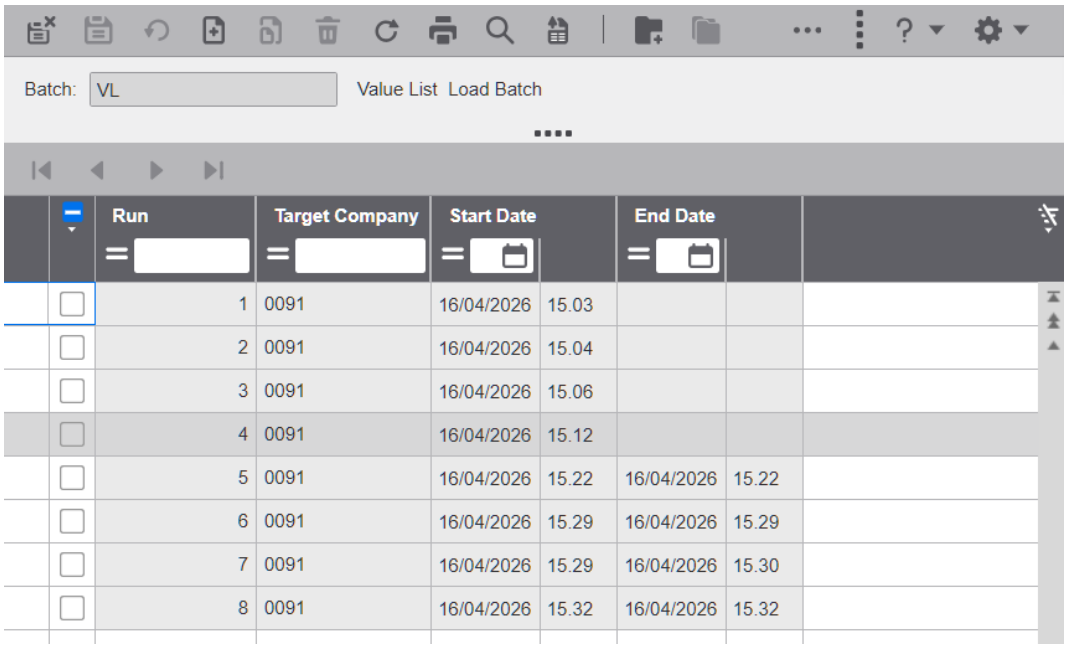


Figura 5.8: Execute Batch Runs (`txamg3200m000`).

Al termine dell'esecuzione, è possibile consultare alcune statistiche come momento di inizio e di fine e i log d'errore per ogni run (Figura 5.9 e 5.10).



Run	Target Company	Start Date		End Date	
1	0091	16/04/2026	15.03		
2	0091	16/04/2026	15.04		
3	0091	16/04/2026	15.06		
4	0091	16/04/2026	15.12		
5	0091	16/04/2026	15.22	16/04/2026	15.22
6	0091	16/04/2026	15.29	16/04/2026	15.29
7	0091	16/04/2026	15.29	16/04/2026	15.30
8	0091	16/04/2026	15.32	16/04/2026	15.32

Figura 5.9: Runs By Batch (`txamg3500m000`).

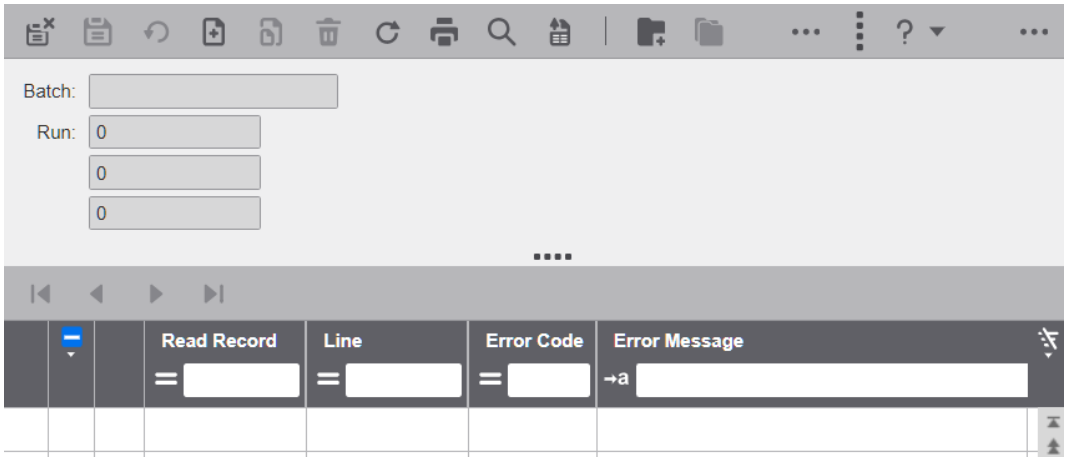


Figura 5.10: Runs Errors (`txamg3503m000`).

5.3.3 Librerie (DLL) e Data Access Layer (DAL)

Tipo	Codice	Descrizione
Libreria	`txamgdll0010`	AlteaIN Migration Tool - DD Integration Library
Libreria	`txamgdll0011`	AlteaIN Migration Tool - Metadata Management & Error Logging
Libreria	`txamgdll0012`	AlteaIN Migration Tool - Script Generator - Load DLL
Libreria	`txamgdll0013`	AlteaIN Migration Tool - Script Generator - Process DLL
DAL	`txamg000`	DAL for txamg000
DAL	`txamg111`	DAL for txamg111
DAL	`txamg203`	DAL for txamg203
DAL	`txamg1110`	Data Files
DAL	`txamg3200`	Execute Batch Runs
DAL	`txamg3500`	Runs by Batch
DAL	`txamg3501`	Runs by Batch/Datafile
DAL	`txamg3502`	Runs by Batch/Datafile/Table
DAL	`txamg3503`	Runs Errors

Tabella 5.1: Elenco delle Librerie (DLL) e DAL sviluppati.

5.3.4 Tabelle di Sistema e Sessioni Operative

Elenco delle principali tabelle (le strutture dettagliate sono consultabili nell'Appendice A) e delle sessioni utilizzate.

Tabella	Descrizione
txamg000	AlteaIN Migration Tool Parameters
txamg100	Table groups definitions
txamg110	Datafiles source definitions
txamg111	DataFile Fields specifications
txamg200	Batch definitions
txamg201	Batch Sequences execution order
txamg202	Target Tables mapping
txamg203	MappedFields specific logic
txamg300	Runs by Batch execution logs
txamg301	Runs by Batch/Datafile detailed logs
txamg302	Runs by Batch/Datafile/Table detailed logs
txamg303	Runs Errors repository

Tabella 5.2: Riepilogo delle tabelle introdotte dal Migration Tool.

Sessione	Descrizione
txamg0100m000	Parameters
txamg1100m000	Datafile Groups
txamg1110m000	Data Files
txamg1111m000	DataFile Fields
txamg2100m000	Batches
txamg2101m000	Batch Sequences
txamg2102m000	Batch Target Tables
txamg2103m000	Table Mappings
txamg3200m000	Execute Batch Runs
txamg3500m000	Runs by Batch
txamg3503m000	Runs Errors

Tabella 5.3: Sessioni principali sviluppate per il Migration Tool.

Dominio	Descrizione
txamg.atyp	DataFile Field Type (Enumerated)
txamg.cbat	Batch (String)
txamg.cdom	Domain (String)
txamg.cmod	Module (String)
txamg.cpac	Package (String)
txamg.cprs	Script/Library Code (String)
txamg.dfil	Data File (String)
txamg.dfnm	Data File Field Name (String)
txamg.fast	ASCII File Format (Enumerated)
txamg.fdnm	Field Name (String)
txamg.fdno	Field Number (Integer)
txamg.flno	Table (String)
txamg.grou	Table Groups (String)
txamg.logn	Login (String)
txamg.mapl	Map Logic (Enumerated)
txamg.migt	Migration Type (Enumerated)
txamg.refm	Reference Mode (Enumerated)
txamg.type	Domain Type (Enumerated)
txamg.utyp	Update Type (Enumerated)

Tabella 5.4: Domini personalizzati tx del Migration Tool.

5.4 Integrazione Documentale

5.4.1 Esportazione delle Specifiche di Migrazione

L'integrazione formale del modulo d'implementazione si traduce nella sua interconnessione con lo strumento aziendale adibito alla documentazione del progetto.

Prima della configurazione ed eventuale esecuzione del batch (le sessioni `txamg2100` e `txamg3200`), l'*utility* estrae il risultato dei mapping salvato nel `txamg203` (Tabella Mappings), formattandolo utilizzando layout predefiniti.

L'esportazione documenta in *Reports* (salvabili in diversi formati come PDF) ogni singolo dominio trasferito, documentando sia i campi copiati senza modifiche (*one-to-one*) (in accordo con `txamg.mapl`) sia quei casi particolari per cui è stato inserito del codice personalizzato, creando le basi per una possibile eventuale manutenzione logica dell'architettura in futuro.

5.5 Ciclo di Vita del Dato

Il processo di migrazione segue un flusso lineare, descritto nel seguente diagramma:

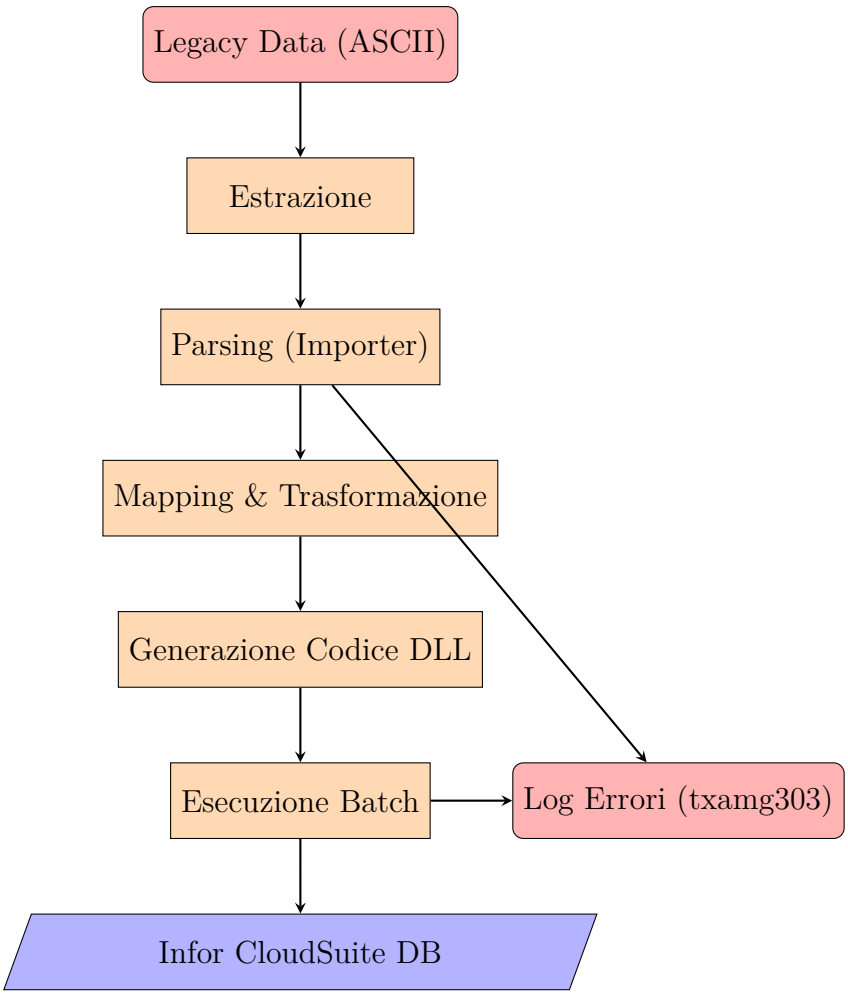


Figura 5.11: Diagramma di Flusso del Ciclo di Vita del Dato.

Il flusso garantisce che ogni record sia tracciato. In caso di errore durante il parsing (formato scorretto) o durante l'esecuzione (vincoli di database), l'*engine* scrive un record dettagliato nella tabella `txamg303`, permettendo la ripartenza della migrazione esattamente da dove si era interrotta.

Capitolo 6

Testing e Validazione

6.1 Strategia di Testing

Testare un software per la migrazione dei dati è stato un aspetto fondamentale del progetto. Data la sensibilità delle logiche del *Data Dictionary* di Infor CloudSuite in ambienti *Cloud Multitenant*, ogni anomalia nel record di inserimento avrebbe gravi conseguenze nel sistema di destinazione.

Inoltre, per il generatore dinamico di codice in Baan C, le procedure di prova hanno incluso non solo l'importer dei dati, ma anche la correttezza sintattica e logica degli *script* generati a *runtime* dalle *DLL* `txamgdll0012` e `txamgdll0013`.

6.1.1 Unit Testing sui Moduli di Parsing

I moduli di base (DLL quali `txamgdll0011`) per l'estrazione e la gestione dei metadata (*metadata management*) degli Infor Exchange sono stati testati su porzioni specifiche di file.

Verificando ogni singola colonna (`txamg1111`), la validazione include il corretto casting di tipi enumerati, decimali a virgola mobile e codifiche in caratteri ASCII, simulando una diversità di input. Durante questa fase è possibile osservare come le diverse versioni del programma non alterino le librerie costruite se le posizioni di inizio (`stpn`) e la lunghezza (`leng`) vengono mantenute.

6.1.2 Test del Flusso di Migrazione

L'integrazione *end-to-end* è validata eseguendo flussi di test governati dalle logiche di *batch* (`txamg2100` e `txamg3200`). Durante questi test, il sistema genera degli *script* di elaborazione e li invia al compilatore interno `bic_compiler`. Se

la generazione avviene correttamente, lo *script* compilato viene agganciato al *batch* ed eseguito.

6.1.3 Test della Sicurezza

Un software di livello *Enterprise* che si interfaccia con il *database* deve essere a prova di anomalie e di sovraccarichi (voluti o semplicemente variazioni impreviste dell'*input* legato all'estrazione da *sistemi legacy*). Oltre a verificare il *mapping* corretto, è stata posta particolare attenzione nelle fasi di validazione e test.

Durante questa fase, sono stati inseriti appositamente nel sistema dei *file* malformati: formati misti in *UTF-8/ISO-8859-1* al posto dello spazio *ASCII* standard, file vuoti e al altri estremamente pesanti per testare la resistenza durante una saturazione del sistema (*Out of Memory*), e misurarne l'utilizzo di memoria *RAM*. Per testare il *logging* delle righe errate, è stato inserito del testo con dimensioni non in linea con i tipi descritti sul file (*es. nome di 50 caratteri su un tipo di lunghezza massima 10*).

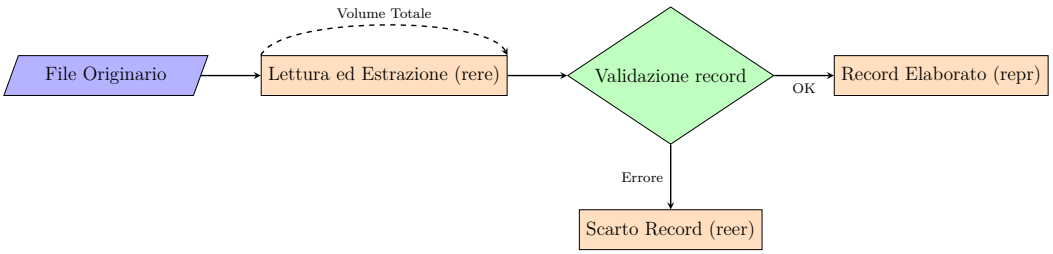


Figura 6.1: Rappresentazione logica del matching numerico di integrità

Un aspetto rilevante del testing d'integrazione riguarda la misurazione dei tempi di *run*. Durante le simulazioni, con *recordset* di piccole (circa 1.000 inserimenti) e medie dimensioni (sino a 100.000 righe su diversi file e tabelle), sono state effettuate delle misurazioni sulle latenze introdotte dal caricamento *cloud* e dai vincoli di referenzialità. In parallelo, per assicurare le prestazioni e le procedure di *rollback*, qualora ci siano rallentamenti o blocchi a livello di rete, il motore transazionale LN è stato istruito affinché le *commit* avvengano a blocchi così come stabilito del numero di record.

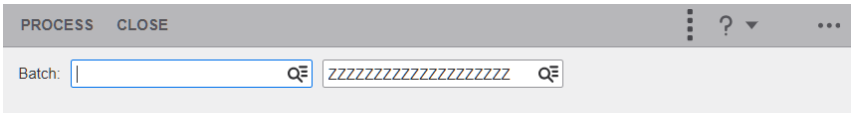


Figura 6.2: Sessione di Esecuzione Batch (txamg3200m000)

I *Runs Errors* offrono un riscontro immediato di blocchi logici o semantici. Sono stati eseguiti scenari in cui la validazione è forzata per respingere record privi di riferimenti chiave, verificando che la linea errata fosse elaborata senza però interromperne il caricamento dell'intero *Run*.

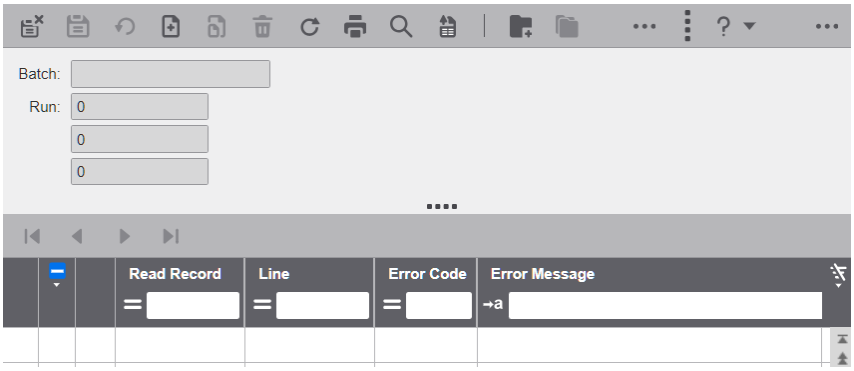


Figura 6.3: Dettaglio degli errori di migrazione (txamg3503m000)

Gli errori segnalati sono dettagliati all'interno della tabella `txamg303`, le cui colonne principali permettono di investigare quale linea e record abbia prodotto l'errore:

- `runs`: Identificativo del Run
- `rere`: Riga di lettura (*Read Record*)
- `line`: Riga testuale all'interno del file sorgente
- `errc`: Codice specifico dell'errore (*Error Code*)
- `errm`: Descrizione del problema riscontrato dal DAL (*Error Message*)

Oltre ai controlli manuali, questo tracciamento funge da base per le strategie di *retry* o per i *sanity check* del dato prima della re-immissione: individuando la tipologia di errore, il consulente o l'operatore può decidere se modificarne il *mapping*, apportare modifiche sul file d'origine o creare estensioni specifiche ai moduli generati dal *tool*.

6.2 Verifica dell'Integrità dei Dati

6.2.1 Matching tra Dati Sorgente (Legacy) e Destinazione (Cloud)

Ogni file migrato viene tracciato sulle tabelle atte al monitoraggio (`txamg301`, `txamg302`). Il *matching* prevede che vengano comparati il numero dei record letti, processati ed eventualmente respinti rispetto all'origine. Questo consente di ottenere la garanzia del risultato atteso, in quanto una discrepanza nel numero di dati letti e scritti, segnalerebbe un errore evidente.

Nello specifico la tabella `txamg301` incapsula le metriche di base del processo:

Campo	Nome	Descrizione
`rere`	Records Read	Il volume totale delle istruzioni e record processati dal tool
`repr`	Record Processed	Quanti dei record immessi hanno dato esito positivo dopo tutti i trigger
`reer`	Records Discarded	Numero di istanze rifiutate o bloccate dal check del DB

Tabella 6.1: Struttura dei conteggi per l'affidabilità (txamg301)

I report prodotti (tramite moduli ad hoc come `txamgdll0011` per il log d'errore) forniscono la garanzia numerica dell'operazione, permettendo al team di validazione di avere stime certe prima dell'import su CloudSuite finale.

Questo approccio garantisce che le disconnessioni di rete o i blocchi temporanei non lascino i dati in uno stato di incertezza. Il tool si comporta in maniera deterministica e produce riscontri sempre validati rispetto al totale.

6.3 Analisi delle Prestazioni e ROI

6.3.1 Benchmark delle Prestazionali

Per valutare l'effettivo miglioramento apportato dall'architettura del tool rispetto ai caricamenti manuali o ai vecchi script, sono stati condotti dei benchmark

basati su dataset crescenti (da 1.000 a 100.000 record). Il grafico in Figura 6.4 mostra i tempi di elaborazione misurati, evidenziando il netto miglioramento garantito dall'elaborazione in batch a blocchi e l'eliminazione dei colli di bottiglia legati alle singole commit di transazione.

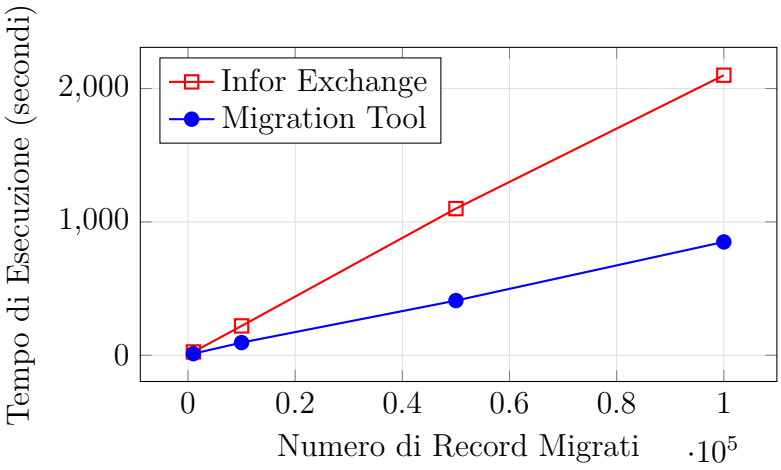


Figura 6.4: Benchmark prestazionale: Confronto tempi di elaborazione.

6.3.2 Analisi degli Errori

Durante i test prestazionali, i *record* scartati (*Discarded Records*) sono stati analizzati e categorizzati per capirne l'origine. La Figura 6.5 illustra la distribuzione percentuale dei fallimenti più comuni.

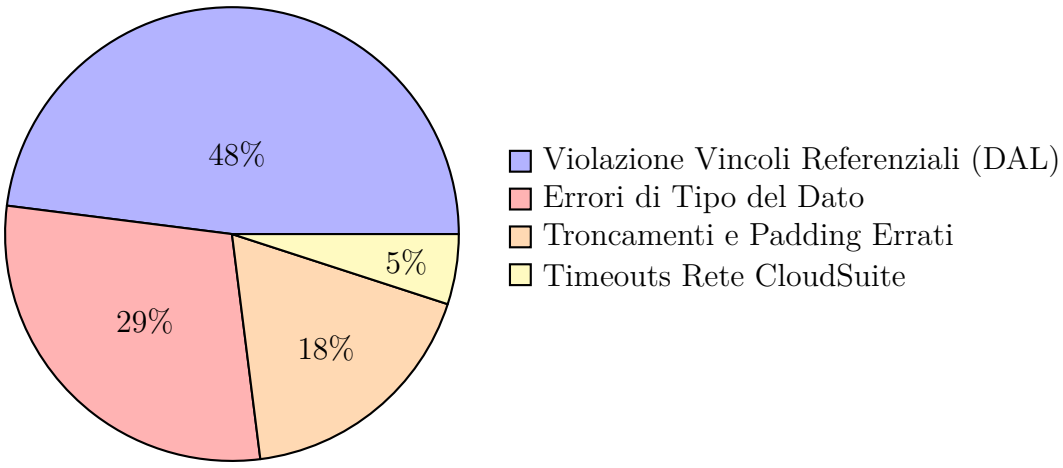


Figura 6.5: Distribuzione delle casistiche d'errore durante i run.

6.3.3 Return on Investment (ROI)

Infine, dal punto di vista gestionale in AlteaIN, l'automazione dei test e del ciclo di migrazione e sviluppo determina un importante risparmio per i consulenti, il quale si tramuta in un certo *ROI* per l'azienda.

La seguente tabella stima il risparmio in giornate, dei dipendenti, rispetto alle fasi operative:

Fase di Migrazione	**Giornate (Legacy)**	**Giornate (Migration Tool)**	**Risparmio**
Analisi e Mapping	5	2	-60%
Sviluppo Script / Generazione Codice	12	1	-91%
Esecuzione Test e Validazione	4	2	-50%
Fix Anomalie	3	1	-66%
Totale stimato	**24**	**6**	**-75%**

Tabella 6.2: Stima della ROI (Return on Investment) per un tipico mapping tabellare di complessità media.

Per evidenziare questo risparmio a livello visivo, è stato realizzato un modello (Figura 6.6) che confronta le tempistiche delle singole fasi.

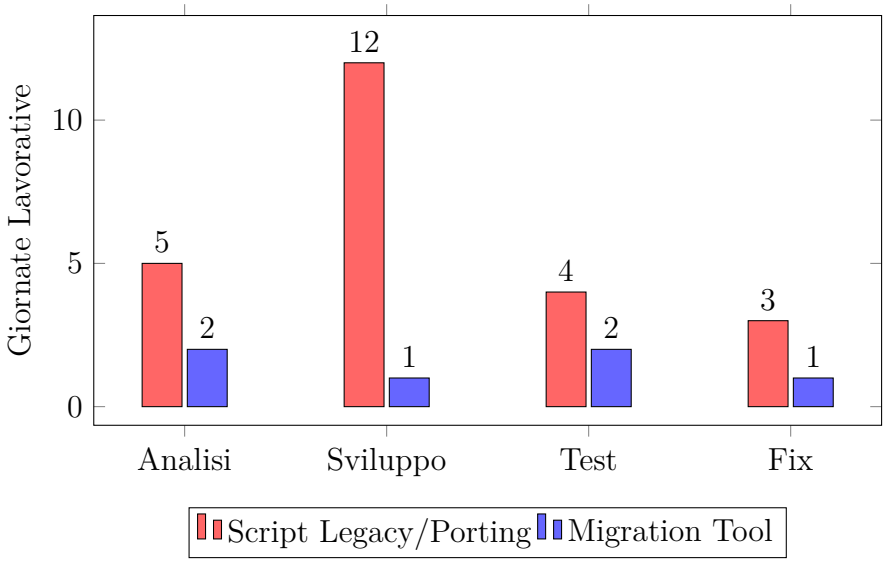


Figura 6.6: Confronto grafico delle risorse impiegate tra vecchio e nuovo modello.

Il drastico taglio dei tempi dedicati allo sviluppo degli script deriva totalmente dalla natura del *Metaprogramming* studiata nell'Applicazione che riduce 12

giorni di interazione Infor a circa 1 giorno. Ciò permette un margine di profitto notevolmente vantaggioso: se ipotizzassimo a 100 giorni (circa 800 ore di lavoro) il quantitativo aziendale investito per la completa stesura del piano e del *tool*, dal risparmio in ogni progetto (18 giornate) AlteaIN ripagherà interamente tale sforzo in tempi molto brevi, trasformando in utile netto le successive migrazioni/*porting* per tutti gli sviluppi e aggiornamenti che seguiranno, dimostrandone la superiorità dei processi *Agile*.

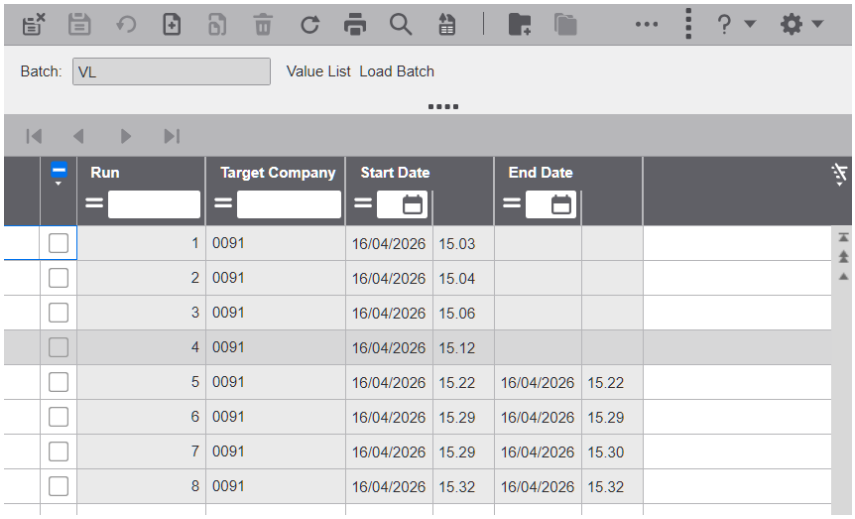


Figura 6.7: Storico e validazione delle esecuzioni (txamg3500m000)

Capitolo 7

Conclusioni

7.1 Obiettivi e Risultati Raggiunti

Nel percorso di questa tesi, è stato analizzato, implementato e documentato un completo *Migration Tool* (sul pacchetto aziendale `txamg`) per Infor LN. Il progetto presentava requisiti complessi, nati per la maggior parte dalla natura *multitenant* dei nuovi ambienti *CloudSuite*, che sono stati affrontati con un approccio parametrico e automatizzato.

Lo sviluppo di librerie capaci di generare dinamicamente codice leggendo il dizionario (come `txamgdll0012` e `txamgdll0013`) e di un *importer* capace di decifrare nativamente i formati Infor *Exchange*, ha permesso di soddisfare l'obiettivo aziendale richiesto. In questo modo si sono potuti garantire buoni risultati a livello di affidabilità e *performance*, alleggerendo notevolmente i consulenti dal frustrante e ripetitivo compito della migrazione (noto internamente come *porting*) manuale, che spesso si traduceva in moltissimi tempi morti e grandi *troubleshooting*.

7.2 Sfide Incontrate

Alcune delle difficoltà che hanno caratterizzato il tirocinio, sono nate dal dover imporre un *mapping* ben sviluppato, lavorando in gran parte su un sistema chiuso (proprietario Infor), in cui l'utilizzo esclusivo dei linguaggi Baan C (3GL e 4GL) ha rappresentato un vincolo tecnico ineliminabile, insieme a standard poco documentati e non intuitivi.

Lavorando in questo ecosistema, avere la possibilità di interfacciarsi con il compilatore interno (e validare il codice *target* a *runtime* su ogni specifico *tenant*) è stato molto importante dal punto di vista dell'analisi e *debug*. Tuttavia, standardizzare e semplificare formati diversi del mondo *legacy* ha imposto la

creazione di strumenti di *parsing* e configurazione di *Runs batch* piuttosto complessi per i vari moduli, compreso, per esempio, il modulo `txamg200`.

Durante la fase di studio dei requisiti e sviluppo, è stato necessario affrontare una fase di *training* inizialmente di base e successivamente più avanzata dei *tool* Infor, quali struttura, nomenclatura, linguaggi, standard, *best practices*, logica di funzionamento, architettura come *middleware* ION, BOD, API e *web service* con formati XML *legacy* come SOAP (non REST), in tempi molto brevi rispetto a quanto normalmente previsto (diversi mesi), al fine di completare il progetto in tempo.

Si fa anche notare la natura molto restrittiva dell'ERP Infor, che non permette l'accesso nemmeno ad amministratori di sistema a parti di API o funzionalità che semplificherebbero lo sviluppo, oltre alla mancata documentazione delle stesse in molti casi a causa di transizioni a più alto livello aziendale, costringendo spesso lo sviluppatore finale a esplorare soluzioni tramite logiche di *trial-and-error*, spesso non *standard*, ove possibile. Questo ha allungato di molto i tempi di sviluppo rispetto a quanto inizialmente previsto, complicandone il contesto generale. Sicuramente, ha dato al sottoscritto un'importante esperienza costruttiva, atta a essere riutilizzata nel tempo per riconoscere e decidere nuove opportunità lavorative, come comportarsi di fronte a colleghi, superiori e *manager*, nonché interfacciarsi con gli strumenti offerti da un'azienda con una struttura federativa più o meno complessa e varia.

7.3 Sviluppi Futuri

Nonostante lo *standard* raggiunto, sarebbe possibile evolvere ulteriormente il *tool* in diverse modi.

In primo luogo, lo schema architetturale per le estensioni *custom* potrebbe venire affiancato da procedure di *Machine Learning* (addestrate sui *mapping legacy*) in grado di suggerire i *mapping* e i moduli di conversione per grandi volumi di tabelle non ancora definite (senza l'*input* umano in interfacce e Sessioni `txamg2103`). Inoltre, la visualizzazione delle specifiche tramite esportazione a modelli *UML* o grafi digitali, se automatizzata nativamente dal *framework*, fornirebbe reportistica *real-time* durante la fase di studio del cliente prima dell'operazione di caricamento, riducendo di fatto tempi e costi e supportando

i consulenti AlteaIN, funzionali e Infor verso transizioni *Cloud* più efficaci e semplici.

Un'altra possibilità riguarda l'integrazione di *Continuous Integration / Continuous Deployment* (*CI/CD*). Attualmente, tutto lo sviluppo dipende dal *repository* interno dell'IDE (LN Studio) e *script* Infor proprietari. Lo sviluppo ed interfacciamento con Git o *versioning code repository* (*VCR*) moderni (come Github/GitLab/Azure DevOps) attraverso l'implementazione di *CI/CD* nativa su Infor Tools consentirebbe ad un *team* di DevOps di monitorare lo stato degli *unit test* e automatizzare i rilasci delle logiche d'esportazione su diverse macchine virtuali e *tenant* aziendali senza richiedere l'uso costante di documentazione e controlli *ad-hoc* del *manager* del progetto.

Appendice A

Dettaglio della Struttura delle Tabelle

Viene qui riportata la struttura dettagliata delle tabelle progettate, sviluppate e utilizzate all'interno del Migration Tool. Ogni tabella è identificata da un codice univoco con prefisso `txamg`. Ove le prime due lettere (`tx`) indicano il package *custom*, le successive tre lettere (`amg`) rappresentano il modulo.

A.1 Tabelle di Configurazione e Metadati

Tabella A.1: Struttura della tabella txamg000 (AlteaIN Migration Tool Parameters)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
indt	tc date	UTC Date/Time	22	Introduction Date
dsca	tc mcs.str50m	Multibyte String	50	Description
lopk	tx amg.cpac	String	2	Load Library Package
lomd	tx amg.cmod	String	3	Load Library Module
loff	tc mcs.long	Long	10	Load Library First Free Number
prpk	tx amg.cpac	String	2	Process Library Package
prmd	tx amg.cmod	String	3	Process Library Module
prff	tc mcs.long	Long	10	Process Library First Free Number
bpat	tc path2	String	128	Base Path for Files

Tabella A.2: Struttura della tabella txamg100 (Table groups)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
grou	tx amg.grou	String	20	Table Group
dsca	tc dsca	Multibyte String	30	Description

Tabella A.2: Struttura della tabella txamg100 (Table groups) – Continua

Campo	Dominio	Tipo Dato	Lung.	Descrizione
bpat	tc path2	String	128	Subfolder

Tabella A.3: Struttura della tabella txamg110 (Datafiles)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
grou	tx amg.grou	String	20	Table Group
dfil	tx amg.dfil	String	20	Data File
dsca	tc dsca	Multibyte String	30	Description
fast	tx amg.fast	Enumerated	10	File Type
sepa	tc mcs.str1	String	1	Separator
frzn	tc yesno	Enumerated	5	Frozen
cprs	tx amg.cprs	String	9	Load Library
cpac	tx amg.cpac	String	2	Ref Table Package
cmod	tx amg.cmod	String	3	Ref Table Module
flno	tx amg.flno	String	3	Ref Table
mufi	tc yesno	Enumerated	5	Multifile Datafile
bpat	tc path2	String	128	Subfolder

Tabella A.4: Struttura della tabella txamg111 (DataFile Fields)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
grou	tx amg.grou	String	20	Table Group
dfil	tx amg.dfil	String	20	Data File
fdno	tx amg.fdno	Integer	4	Field Number
fdnm	tx amg.dfnm	String	20	Field Name
dsca	tc dsca	Multibyte String	30	Description
stpn	tx amg.fdno	Integer	4	Start Position
leng	tx amg.fdno	Integer	4	Length
atyp	tx amg.atyp	Enumerated	10	Dafile Field Type
adgb	tc mcs.int	Integer	3	Digits Before
adga	tc mcs.int	Integer	3	Digits After
cpac	tx amg.cpac	String	2	Ref Table Package
cmod	tx amg.cmod	String	3	Ref Table Module

Tabella A.4: Struttura della tabella txamg111 (DataFile Fields) – Continua

Campo	Dominio	Tipo Dato	Lung.	Descrizione
flno	tx amg.flno	String	3	Ref Table
rfnm	tx amg.fdnm	String	8	Ref Field Name

A.2 Tabelle di Mapping e Logica

Tabella A.5: Struttura della tabella txamg200 (Batch)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
dsca	tc dsca	Multibyte String	30	Description
bpat	tc path2	String	128	Subfolder

Tabella A.6: Struttura della tabella txamg201 (Batch Sequences)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
sequ	tc mcs.int	Integer	3	Seq.
grou	tx amg.grou	String	20	Table Group
dfil	tx amg.dfil	String	20	Data File
migt	tx amg.migt	Enumerated	10	Migration Type
cprs	tx amg.cprs	String	9	Process Library
ilgn	tx amg.logn	String	8	Internal Owner
elgn	tx amg.logn	String	8	External Owner
txta	tc txtn	Text	0	Notes
frzn	tc yesno	Enumerated	5	Frozen

Tabella A.7: Struttura della tabella txamg202 (Target Tables)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
sequ	tc mcs.int	Integer	3	Seq.
sseq	tc mcs.int	Integer	3	SubSeq.
cpac	tx amg.cpac	String	2	Package

Tabella A.7: Struttura della tabella txamg202 (Target Tables) – Continua

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cmod	tx amg.cmod	String	3	Module
flno	tx amg.flno	String	3	Table
flnm	tc dsca	Multibyte String	30	Table Name
ilgn	tx amg.logn	String	8	Internal Owner
elgn	tx amg.logn	String	8	External Owner
shrd	tc yesno	Enumerated	5	Shared
udal	tc yesno	Enumerated	5	Use DAL
utyp	tx amg.utyp	Enumerated	10	DB Operation
txta	tc txtn	Text	0	Functional Notes

Tabella A.8: Struttura della tabella txamg203 (Mapped Fields)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
sequ	tc mcs.int	Integer	3	Seq.
sseq	tc mcs.int	Integer	3	SubSeq.
cpac	tx amg.cpac	String	2	Package
cmod	tx amg.cmod	String	3	Module
flno	tx amg.flno	String	3	Table
fdno	tx amg.fdno	Integer	4	Field N.
fdnm	tx amg.fdnm	String	8	Field Name
icdf	tc yesno	Enumerated	5	CDF
isml	tc yesno	Enumerated	5	ML
fdsc	tc dsca	Multibyte String	30	Field Desc
comb	tc yesno	Enumerated	5	Comb.
pacd	tx amg.cpac	String	2	Domain Package
cdom	tx amg.cdom	String	12	Domain
type	tx amg.type	Enumerated	10	Data Type
digb	tc mcs.int	Integer	3	Digit Before
diga	tc mcs.int	Integer	3	Digit After
iniv	tc dsca	Multibyte String	30	Initial Value
mand	tc yesno	Enumerated	5	Mandatory
rfpa	tx amg.cpac	String	2	Ref. Table Package

Tabella A.8: Struttura della tabella txamg203 (Mapped Fields) – Continua

Campo	Dominio	Tipo Dato	Lung.	Descrizione
rfmo	tx amg.cmod	String	3	Ref. Table Module
rffn	tx amg.cmod	String	3	Ref. Table
rffd	tx amg.fdnm	String	8	Ref. Field
dept	tc mcs.int	Integer	3	Depth

A.3 Tabelle di Esecuzione e Log

Tabella A.9: Struttura della tabella txamg300 (Runs by Batch)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
runs	tc mcs.long	Long	10	Run
ncmp	tc ncmp	Integer	4	Target Company
stdt	tc date	UTC Date/Time	22	Start Date
endt	tc date	UTC Date/Time	22	End Date

Tabella A.10: Struttura della tabella txamg301 (Runs by Batch/Datafile)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
runs	tc mcs.long	Long	10	Run
sequ	tc mcs.long	Long	10	Sequence
stdt	tc date	UTC Date/Time	22	Start Date
endt	tc date	UTC Date/Time	22	End Date
rere	tc mcs.long	Long	10	Records Read
repr	tc mcs.long	Long	10	Record Processed
reer	tc mcs.long	Long	10	Records Discarded

Tabella A.11: Struttura della tabella txamg302 (Runs by Batch/Datafile/Table)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
runs	tc mcs.long	Long	10	Run

Tabella A.11: Struttura della tabella txamg302 (Runs by Batch/Datafile/Table) – Continua

Campo	Dominio	Tipo Dato	Lung.	Descrizione
sequ	tc mcs.long	Long	10	Sequence
sseq	tc mcs.long	Long	10	Subsequence
stdt	tc date	UTC Date/Time	22	Start Date
endt	tc date	UTC Date/Time	22	End Date

Tabella A.12: Struttura della tabella txamg303 (Runs Errors)

Campo	Dominio	Tipo Dato	Lung.	Descrizione
cbat	tx amg.cbat	String	20	Batch
runs	tc mcs.long	Long	10	Run
sequ	tc mcs.long	Long	10	Sequence
sseq	tc mcs.long	Long	10	Subsequence
rere	tc mcs.long	Long	10	Read Record
line	tc mcs.long	Long	10	Line
errc	tc mcs.int	Integer	3	Error Code
errm	tc mcs.s999m	Multibyte String	999	Error Message

Bibliografia

[1] Juan Pablo Franceschetti Alessio Verazzi Andrea Cerini. *Infor LN - Exchange Guide*. Guida alla configurazione degli schemi di importazione ed esportazione ASCII. Infor LN Exchange Altea IN - Rev.02. 2020 (cit. a p. 10).

[2] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesi di dott. University of California, Irvine, 2000 (cit. a p. 3).

[3] Vinay Kumar Gali e Avneesh Kumar. «Master Data Migration and Integration Strategies for Global Oracle Cloud ERP Deployments». In: *Journal of Emerging Technologies and Innovative Research (JETIR)* 12.1 (2025), b525–b530. ISSN: 2349-5162. URL: `https://www.jetir.org/view?paper=JETIR2501185` (cit. a p. 12).

[4] Infor Development Team. *Infor LN - 3GL and 4GL Programming Guide*. Manuale per lo sviluppo di script e personalizzazioni in linguaggio Baan C. Infor ES Programmers Guide LN 10.8.0 en (progguide_10.8.0_en.chm). 2025 (cit. a p. 9).

[5] Infor Development Team. *Infor LN - Data Access Layer (DAL) Overview and Reference*. Documentazione tecnica per l'implementazione della logica di business e integr` dati. Infor ES Programmers Guide LN 10.8.0 en (progguide_10.8.0_en.chm). 2025 (cit. a p. 9).

[6] Infor Development Team. *Infor LN - Extended Data Access Layer (DAL2) Overview*. Documentazione tecnica sull'evoluzione del framework DAL per l'integrazione, 3GL e 4GL. Infor ES Programmers Guide LN 10.8.0 en (progguide_10.8.0_en.chm). 2025 (cit. a p. 9).

[7] Ralph Kimball e Joe Caserta. *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. John Wiley & Sons, 2004. ISBN: 978-0764567575 (cit. a p. 10).

[8] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2015. ISBN: 978-1292096131 (cit. alle pp. 2, 16).

Sitografia

[9] Kent Beck et al. *Manifesto for Agile Software Development*. Accessed: 2026-04-27. 2001. URL: https://agilemanifesto.org/ (cit. a p. 2).

[10] *CloudSuite™ Industrial Enterprise*. Accessed: 2026-04-27. 2026. URL: https://www.infor.com/it-it/products/cloudsuite-industrial-enterprise (cit. alle pp. 1, 7).

[11] Infor. *Infor Intelligent Open Network (ION)*. Accessed: 2026-04-27. 2026. URL: https://www.infor.com/it-it/platform/ipaas/ion-middleware (cit. a p. 3).

[12] Infor. *What is multi-tenancy in the cloud?* Accessed: 2026-04-28. 2026. URL: https://www.infor.com/platform/what-is-multi-tenancy-in-the-cloud (cit. alle pp. 1, 5).

[13] *Infor LN*. Accessed: 2026-04-27. 2026. URL: https://www.infor.com/it-it/solutions/erp/ln (cit. alle pp. 1, 7).

[14] Oracle NetSuite. *ERP Data Migration Tips and Best Practices*. Accessed: 2026-04-28. 2026. URL: https://www.netsuite.com/portal/resource/articles/erp/erp-data-migration.shtml (cit. a p. 6).

[15] TechTarget. *7 challenges, best practices for ERP data migration*. Accessed: 2026-04-28. 2025. URL: https://www.techtarget.com/searcherp/feature/12-cloud-ERP-data-migration-best-practices (cit. alle pp. 5, 14).